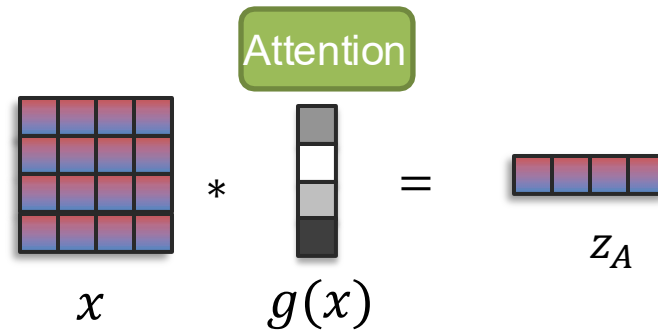


Deep learning

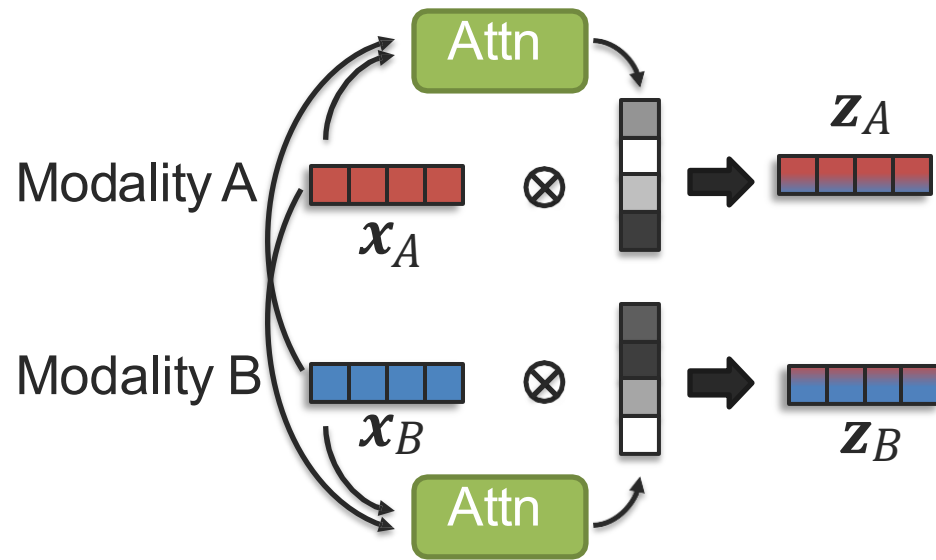
Lecture 8

Last time: Attentive Pooling



$$z_A = x_A * g(x) = \sum [g(x)]_i \cdot x_i; \quad \begin{array}{l} x \in \mathbb{R}^d \times n \\ g(x) \in n \times 1 \end{array}$$

Gated Co-Attention



Gated coordination:

$$z_A = g_A(x_A, x_B)x_a$$

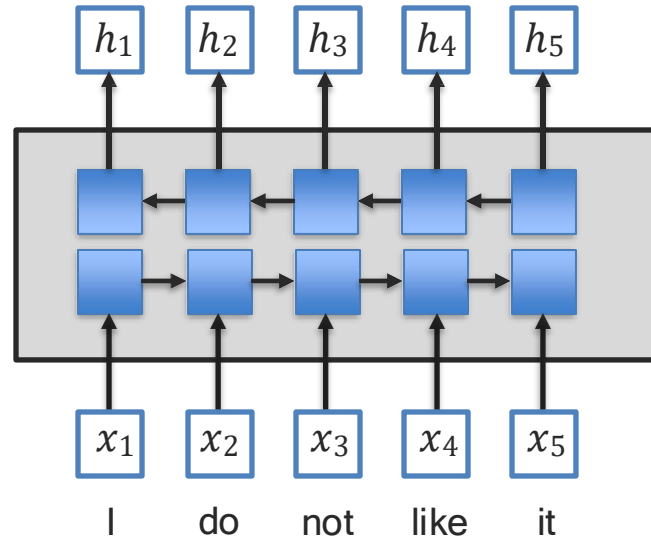
$$z_B = g_B(x_A, x_B)x_b$$

Also known as Co-Attention*

*- High-Order Attention; Schwartz et al.; NIPS'17

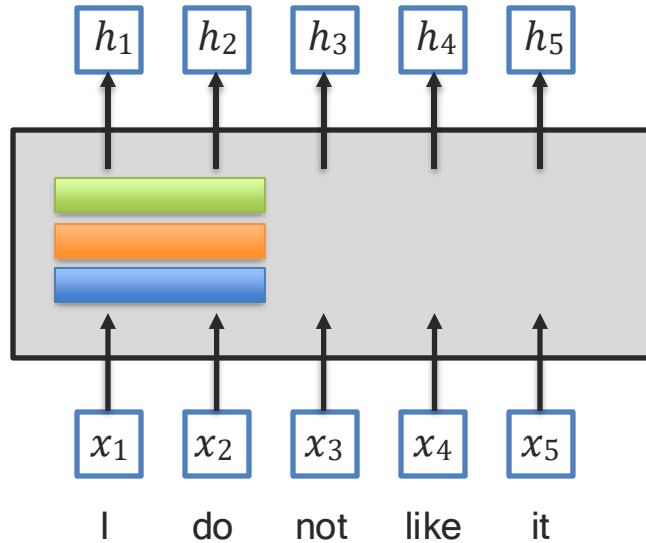
Today: Attention for sequence representation?

Recall: Bi-directional LSTM



But harder to parallelize...

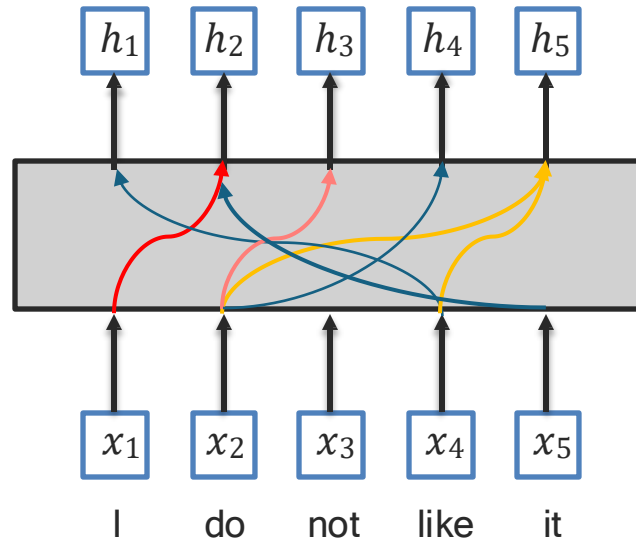
Recall : Convolutions



Can be parallelized!

But modeling long-range dependencies
require multiple layers

Today: Self-attention



Can be parallelized!

Long-range dependencies

Dynamic attention weights

Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

X_1

X_2

X_3

Q_1

Q_2

Q_3

Q_4

Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

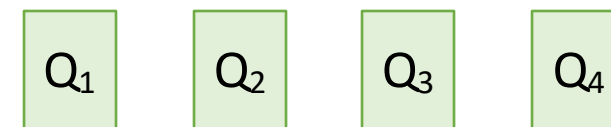
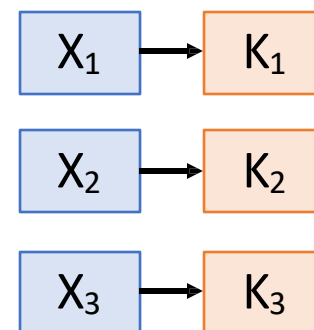
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

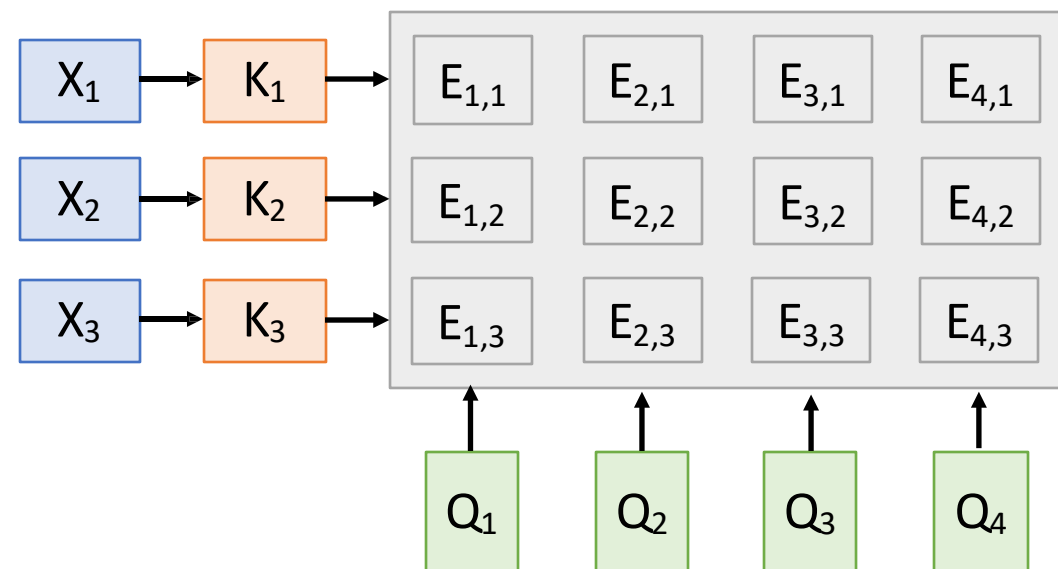
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

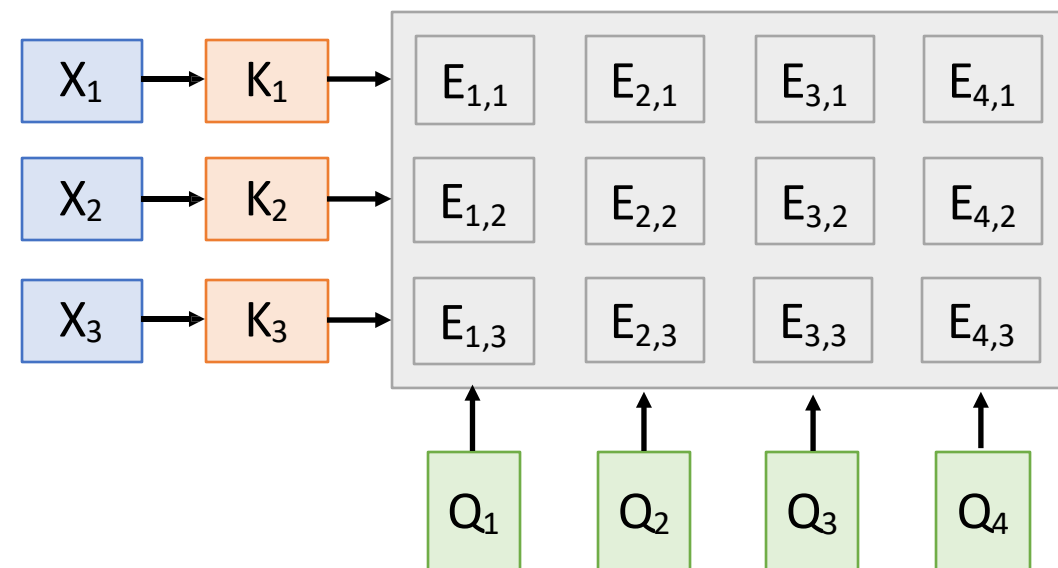
Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

We used the correlation matrix
before for attention pooling

			
What	1.2	2.3	1.7
animal	-0.4	6.5	4.2
is	1.3	1.2	0.6

$A_{1,1}$	$A_{2,1}$	$A_{3,1}$	$A_{4,1}$
$A_{1,2}$	$A_{2,2}$	$A_{3,2}$	$A_{4,2}$
$A_{1,3}$	$A_{2,3}$	$A_{3,3}$	$A_{4,3}$

Softmax($\uparrow$)



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

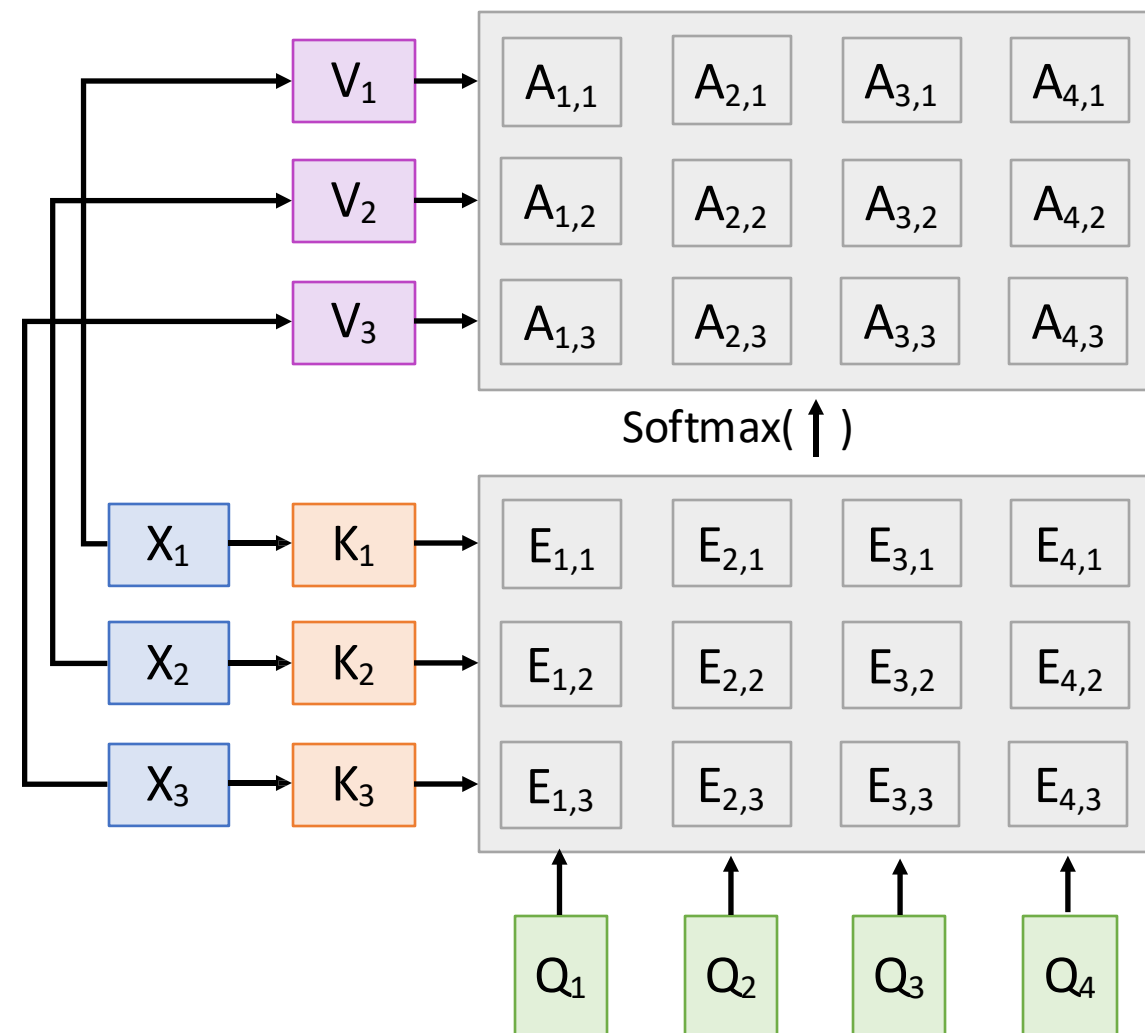
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

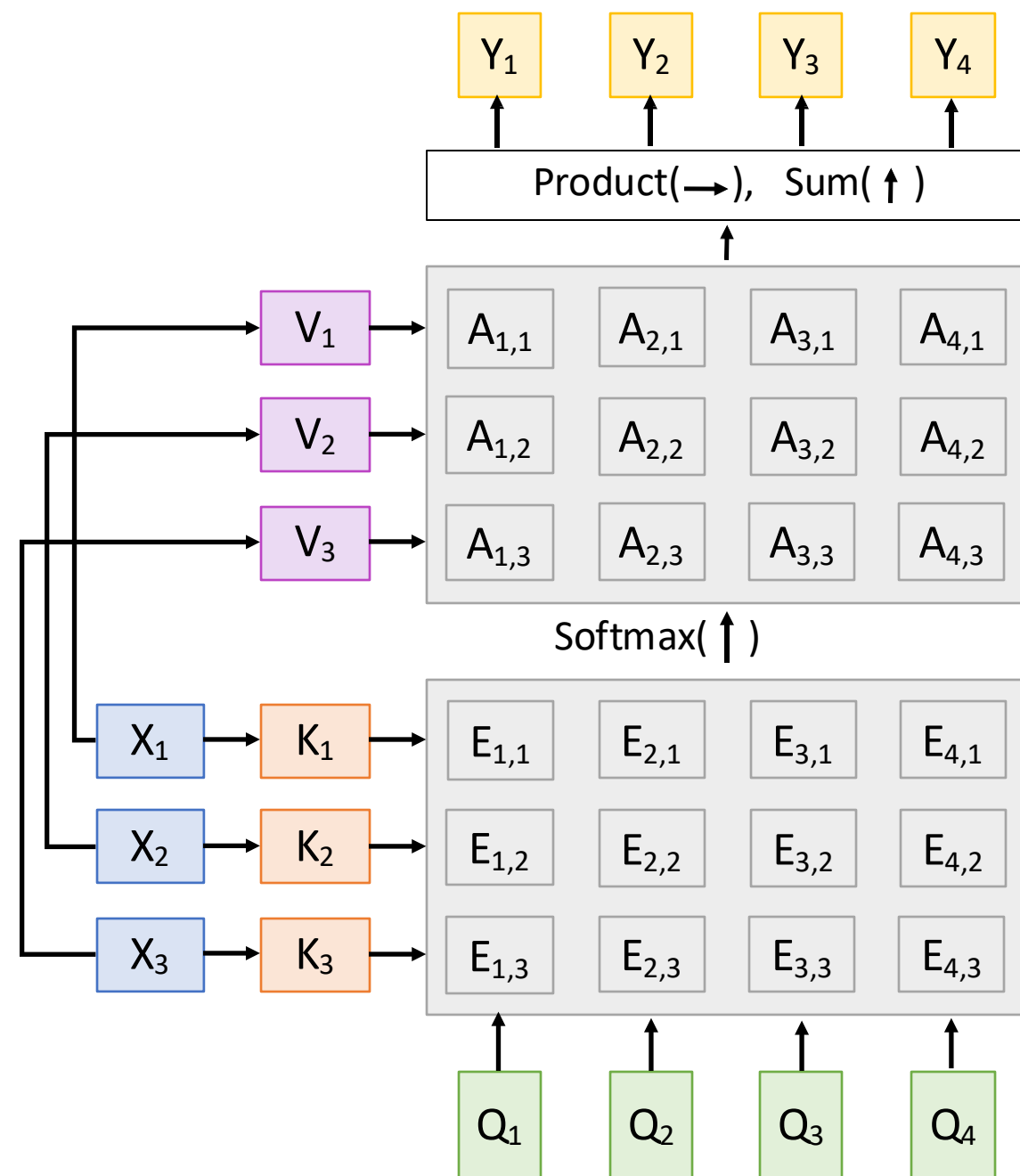
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Intuitively, Attention Transformer Layer

$$\begin{aligned} & \text{softmax} \left(\frac{\begin{matrix} \mathbb{R}^{n_A \times d} \\ Q \end{matrix} \times \begin{matrix} \mathbb{R}^{d \times n_B} \\ K^T \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \mathbb{R}^{n_B \times d} \\ V \end{matrix} \\ &= \begin{matrix} Z \\ \mathbb{R}^{n_A \times d} \end{matrix} \end{aligned}$$

Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:
 $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$

X_1

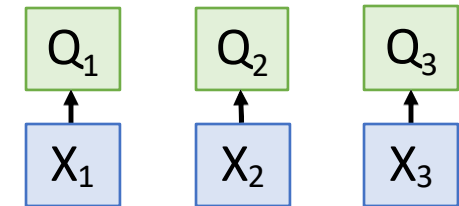
X_2

X_3

Self-Attention Layer

One **query** per **input vector**

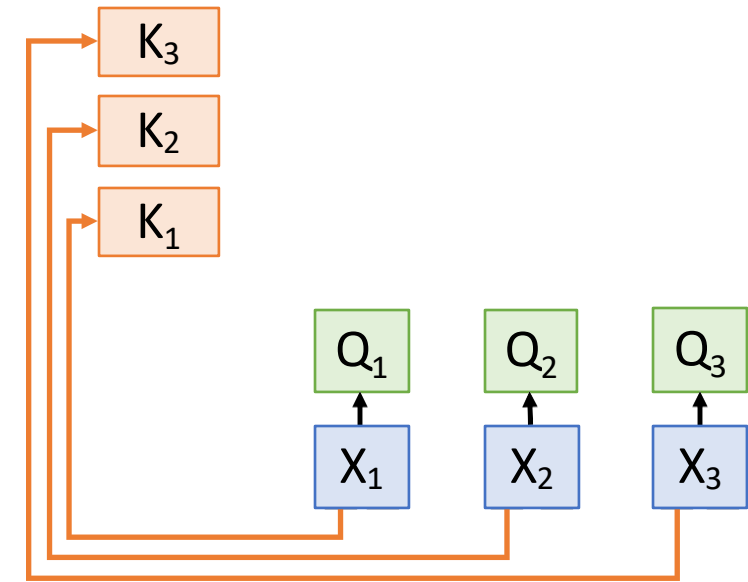
- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

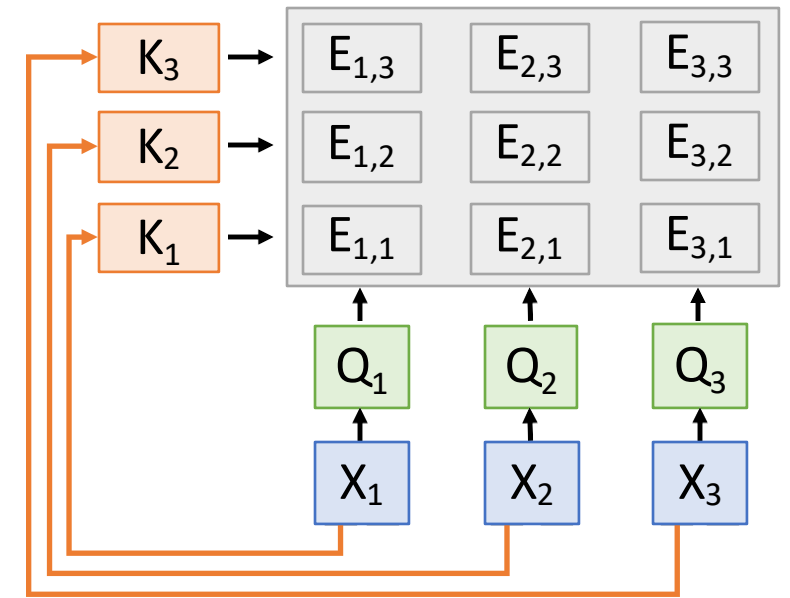
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

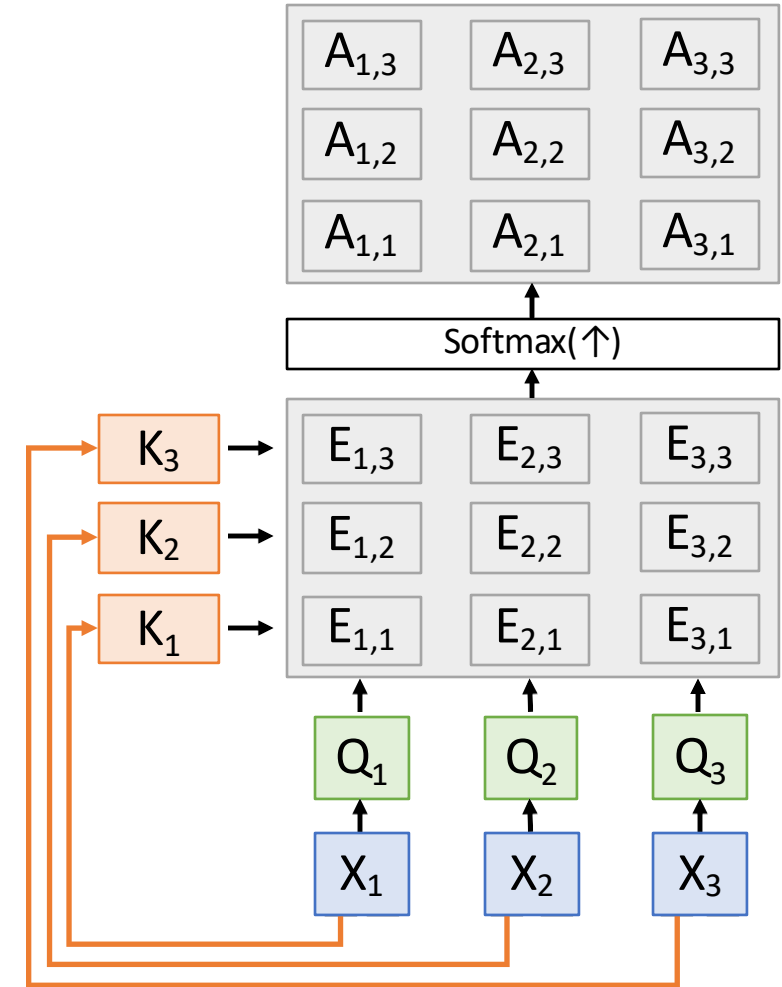
$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:
 $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

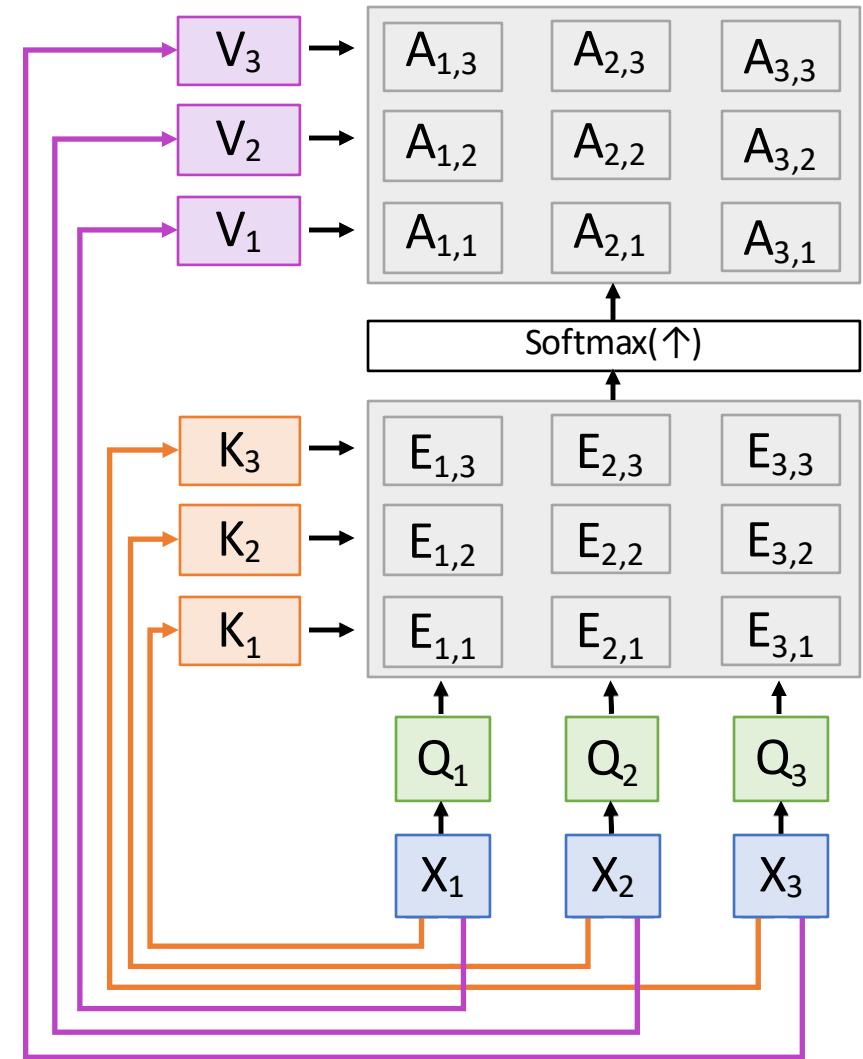
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

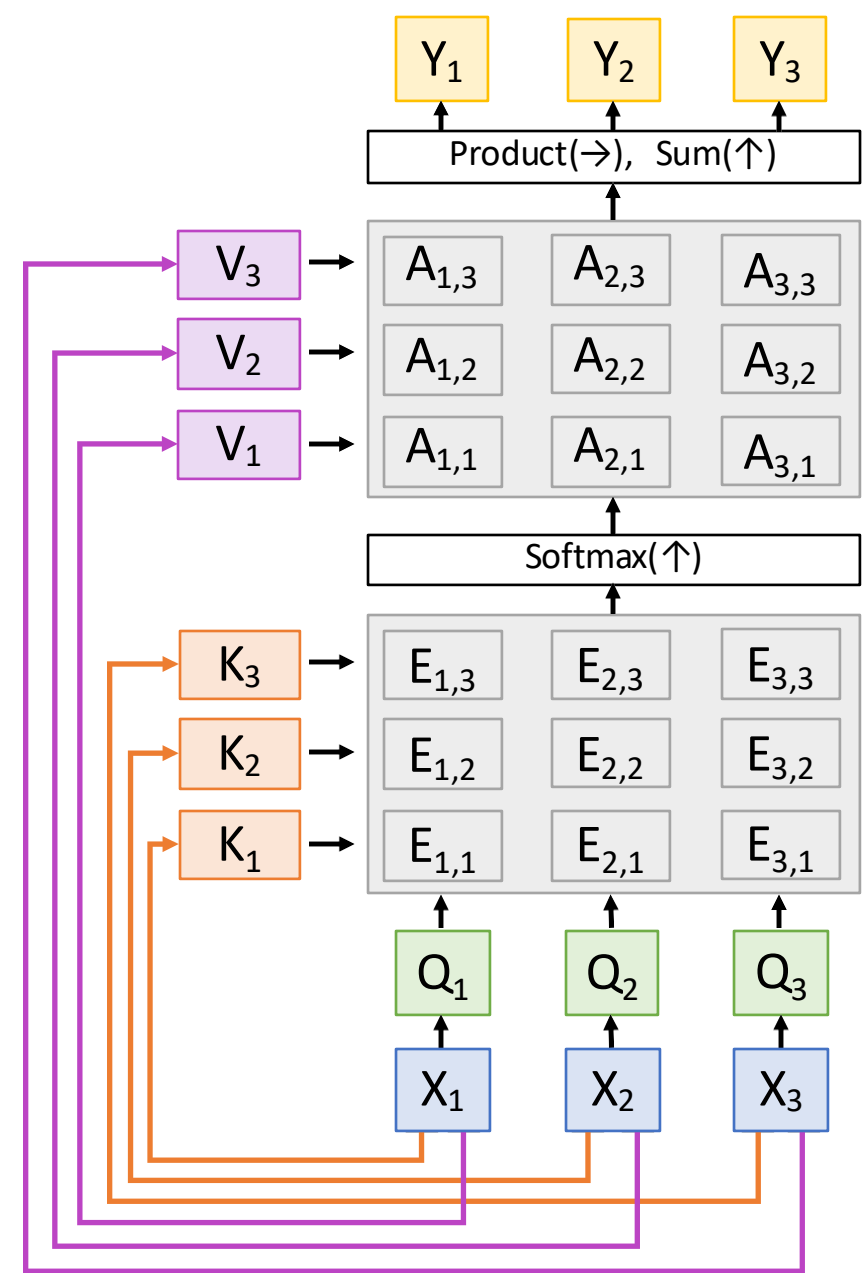
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



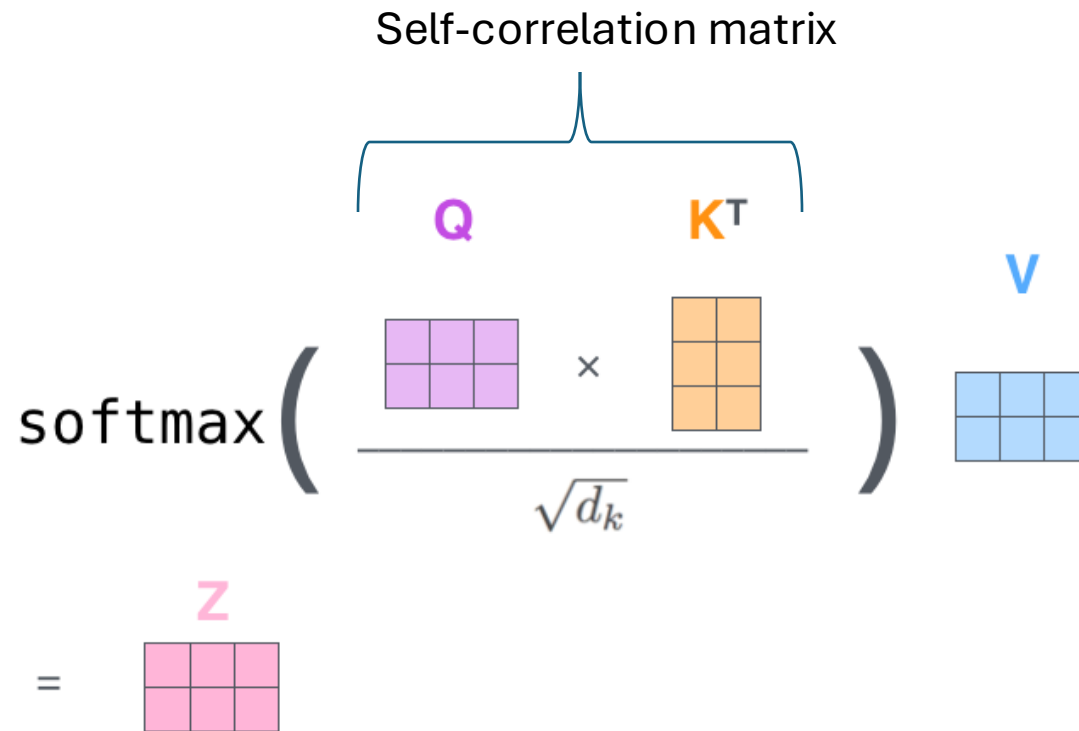
Self-correlation matrix

Correlation Matrix: $\overset{\text{Q}}{\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array}} \times \overset{\text{K}^T}{\begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline \end{array}} \in \mathbb{R}^{n \times n}$

	What	animal	is
What	0.4	0.1	0.1
animal	0.1	0.3	0.1
is	0.1	0.3	0.3

Intuetively, Self-Attention-layer

- Given a sequence $x \in \mathbb{R}^{n \times d}$ (e.g., a sentence)



$$Q = x \cdot W_Q \in \mathbb{R}^{n \times d}$$

$$K = x \cdot W_K \in \mathbb{R}^{n \times d}$$

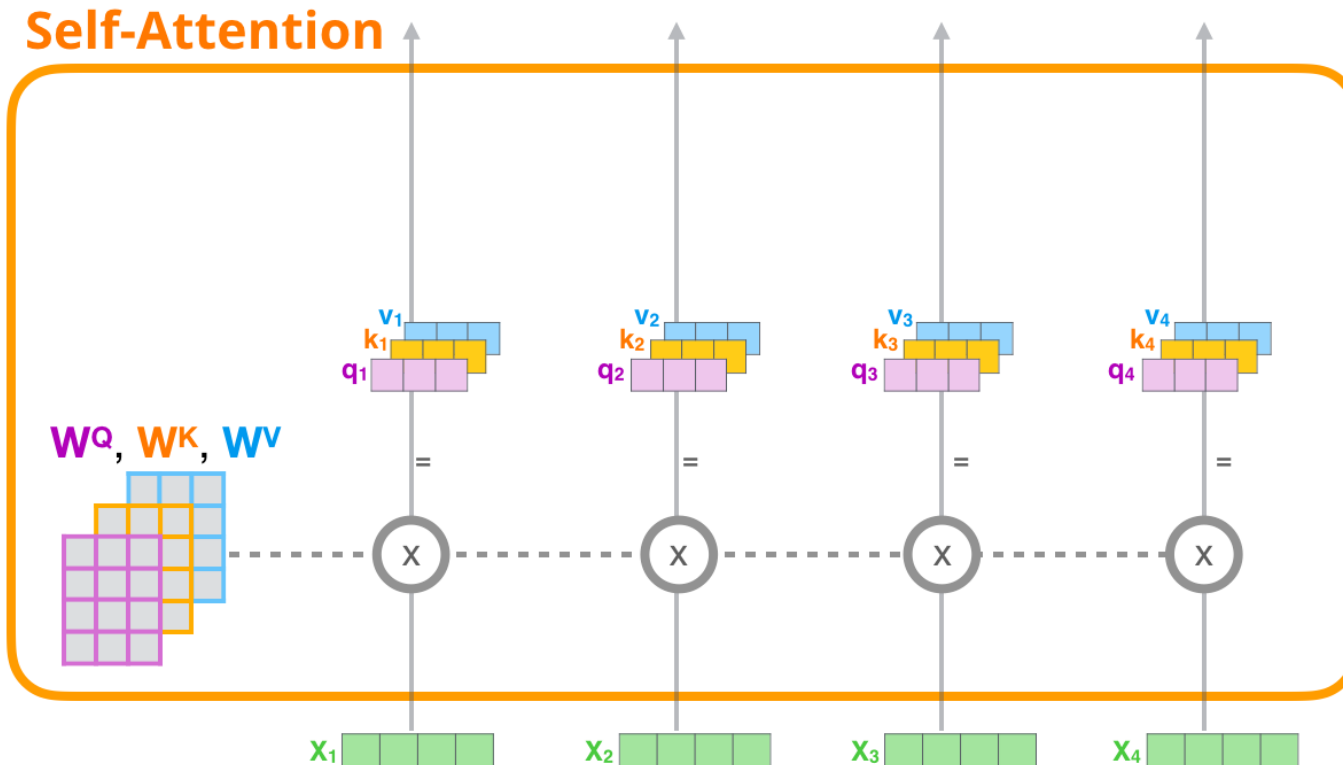
$$V = x \cdot W_V \in \mathbb{R}^{n \times d}$$

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$$

Summary:

Self-Attention-layer

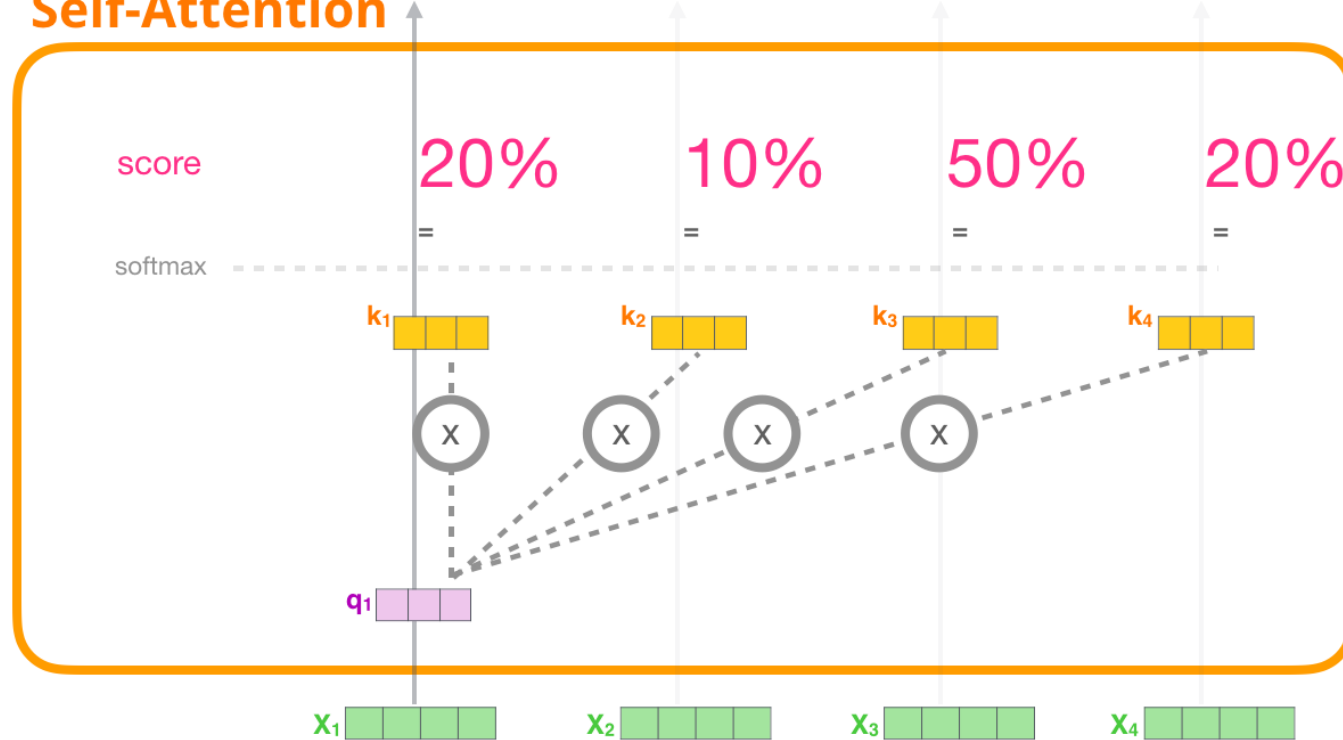
1) For each input token, create a **query vector**, a **key vector**, and a **value vector** by multiplying by weight Matrices W^Q , W^K , W^V



Self-Attention-layer

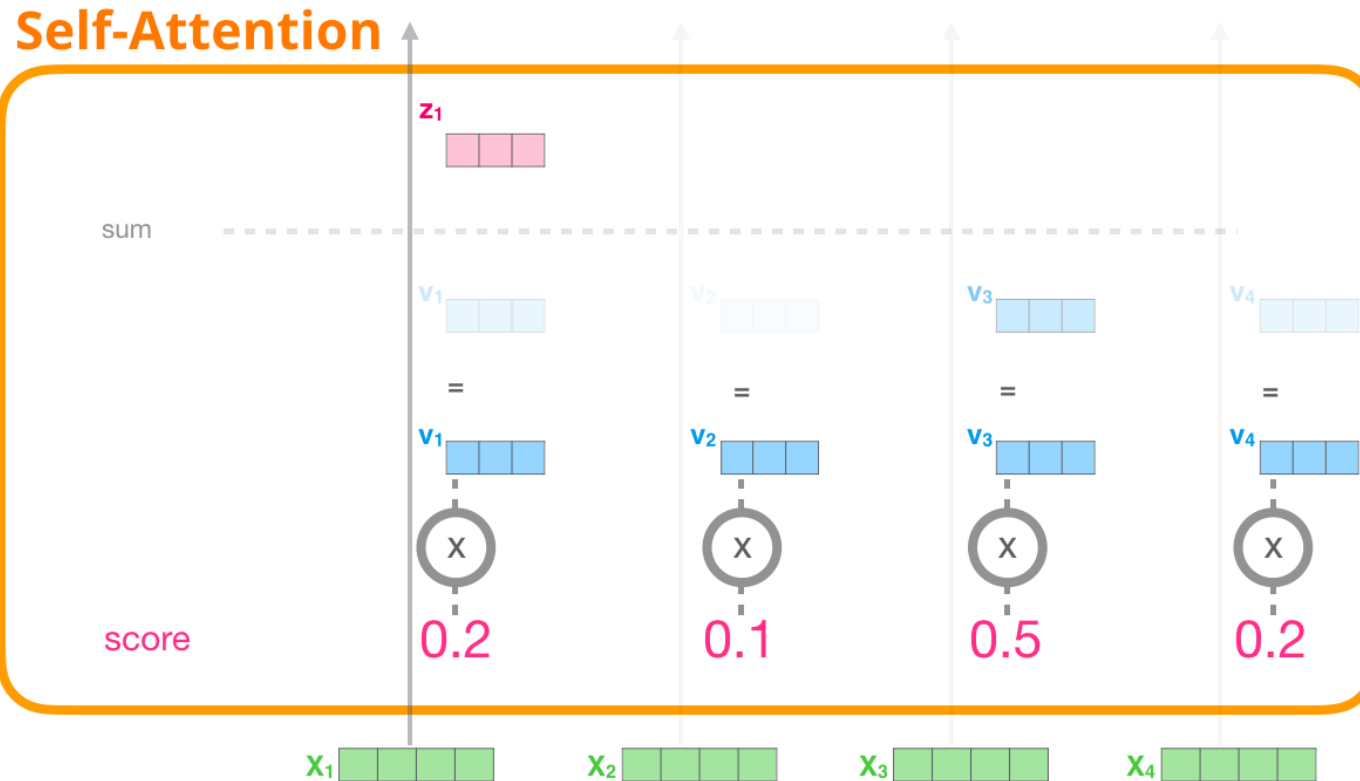
2) Multiply (dot product) the current **query vector**, by all the **key vectors**, to get a score of how well they match

Self-Attention



Self-Attention-layer

3) Multiply the **value vectors** by the **scores**, then sum up



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Consider **permuting**
the input vectors:

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

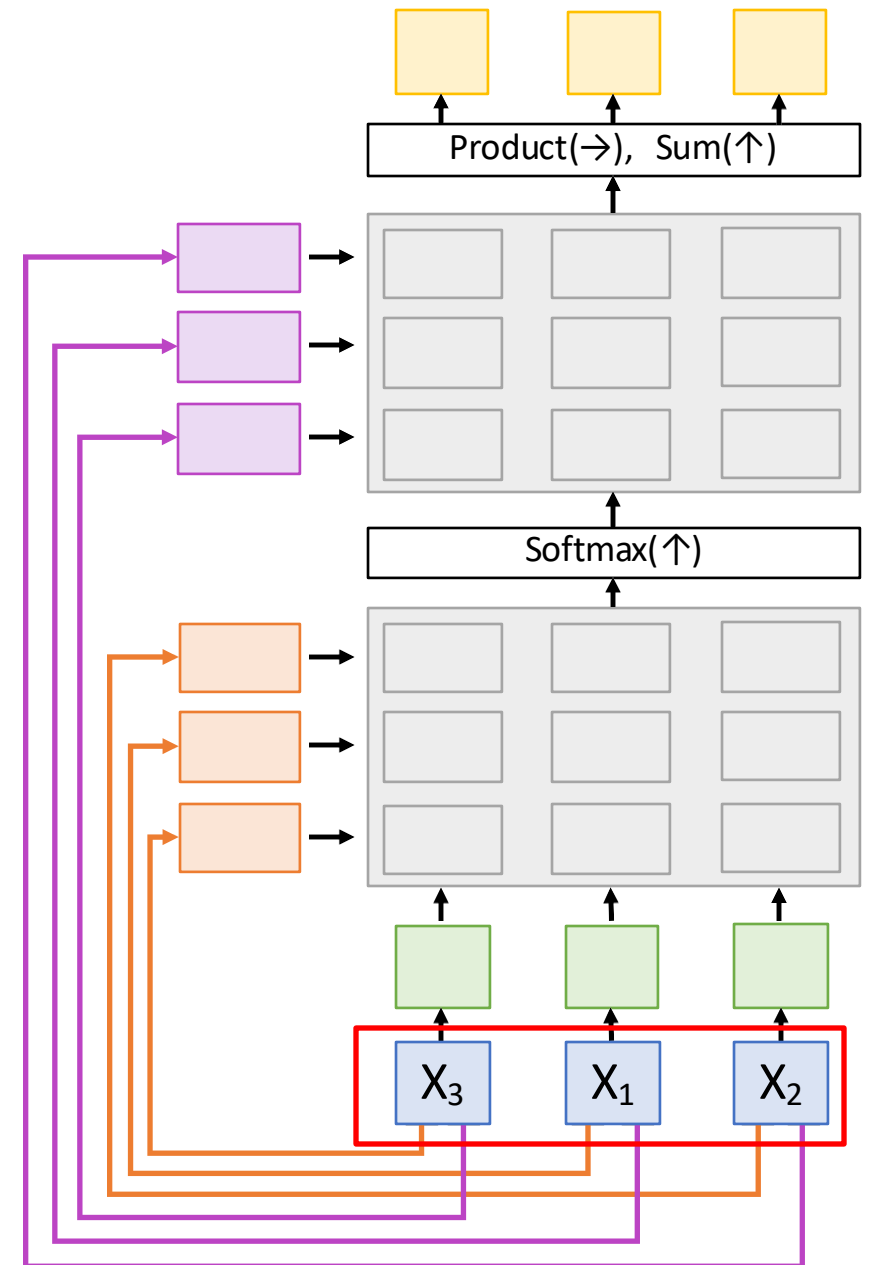
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

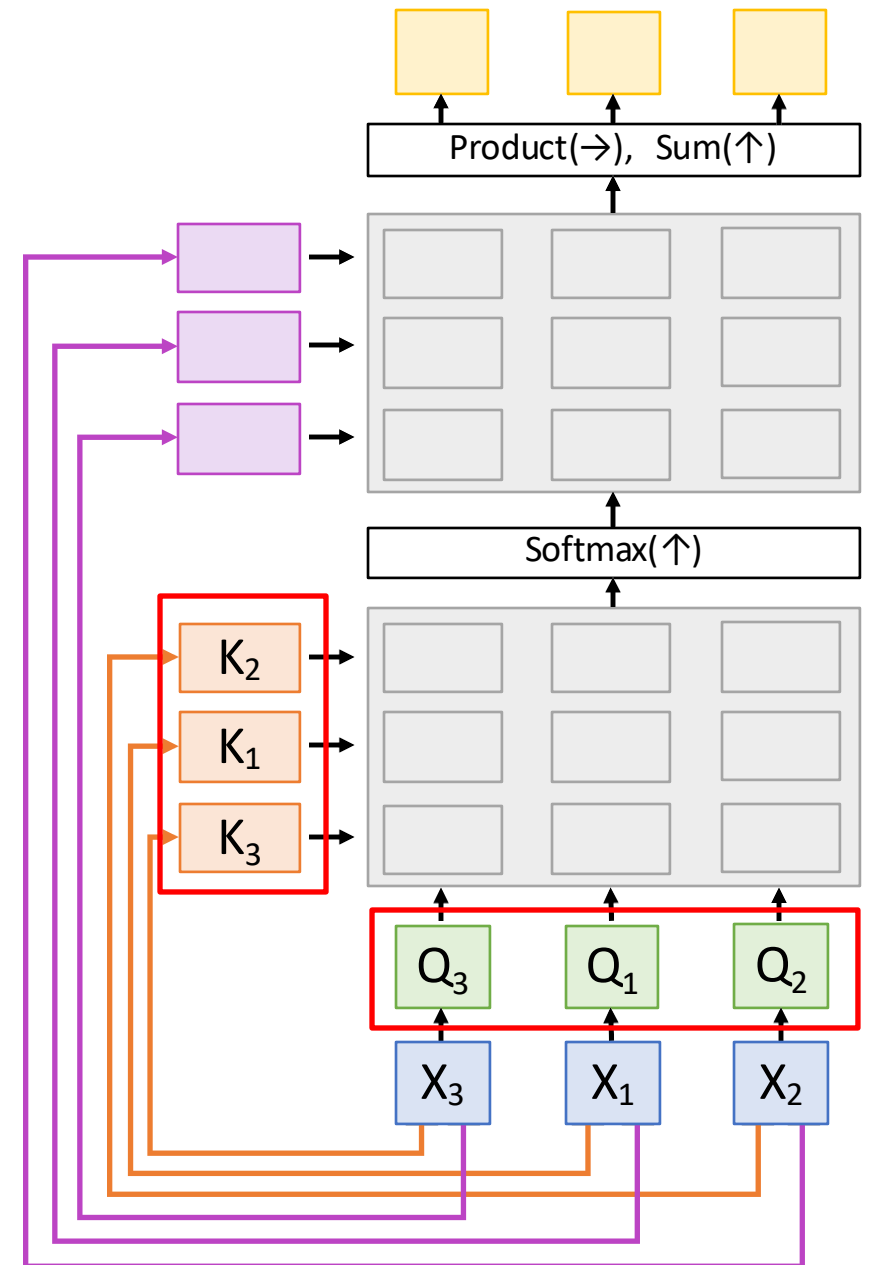
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Queries and Keys will be
the same, but permuted



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

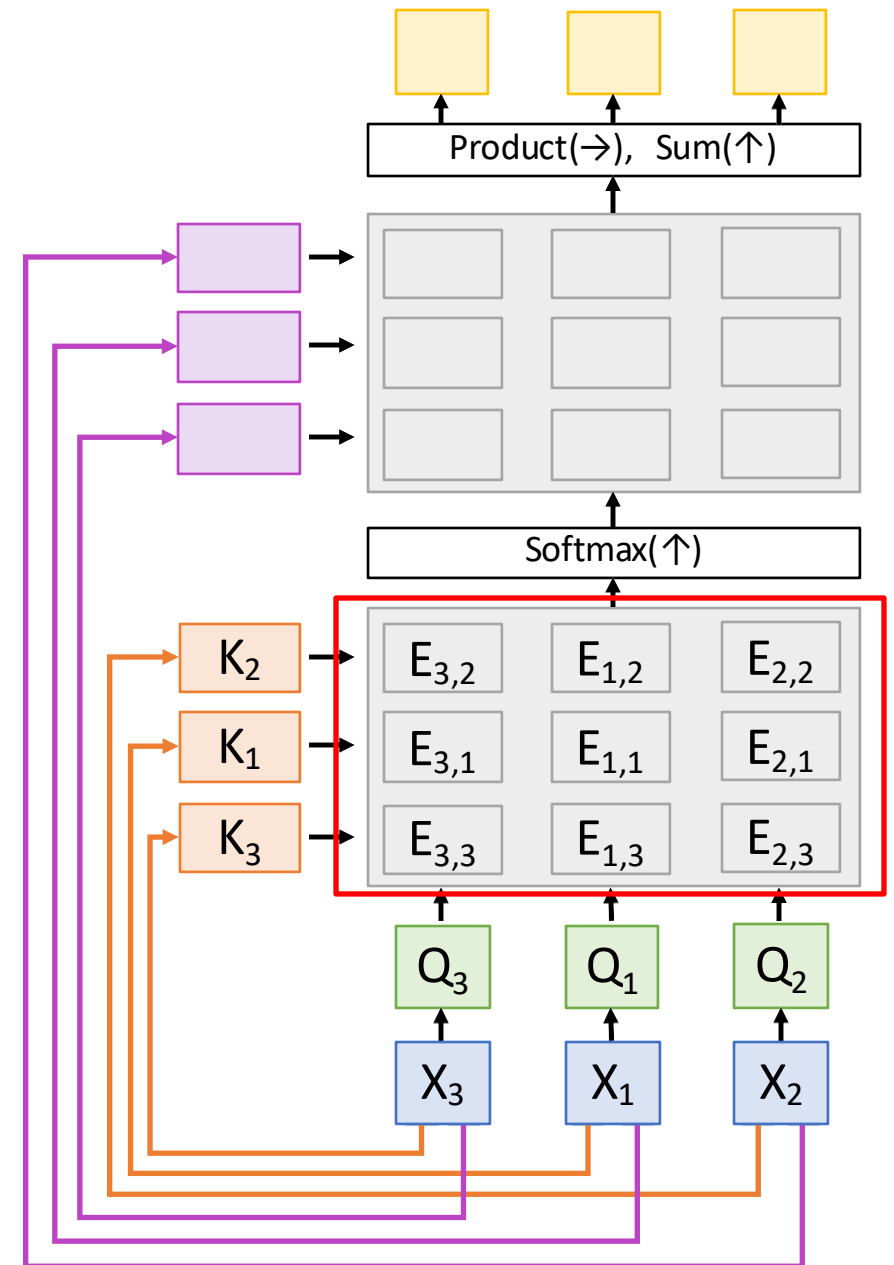
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Similarities will be the
same, but permuted



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

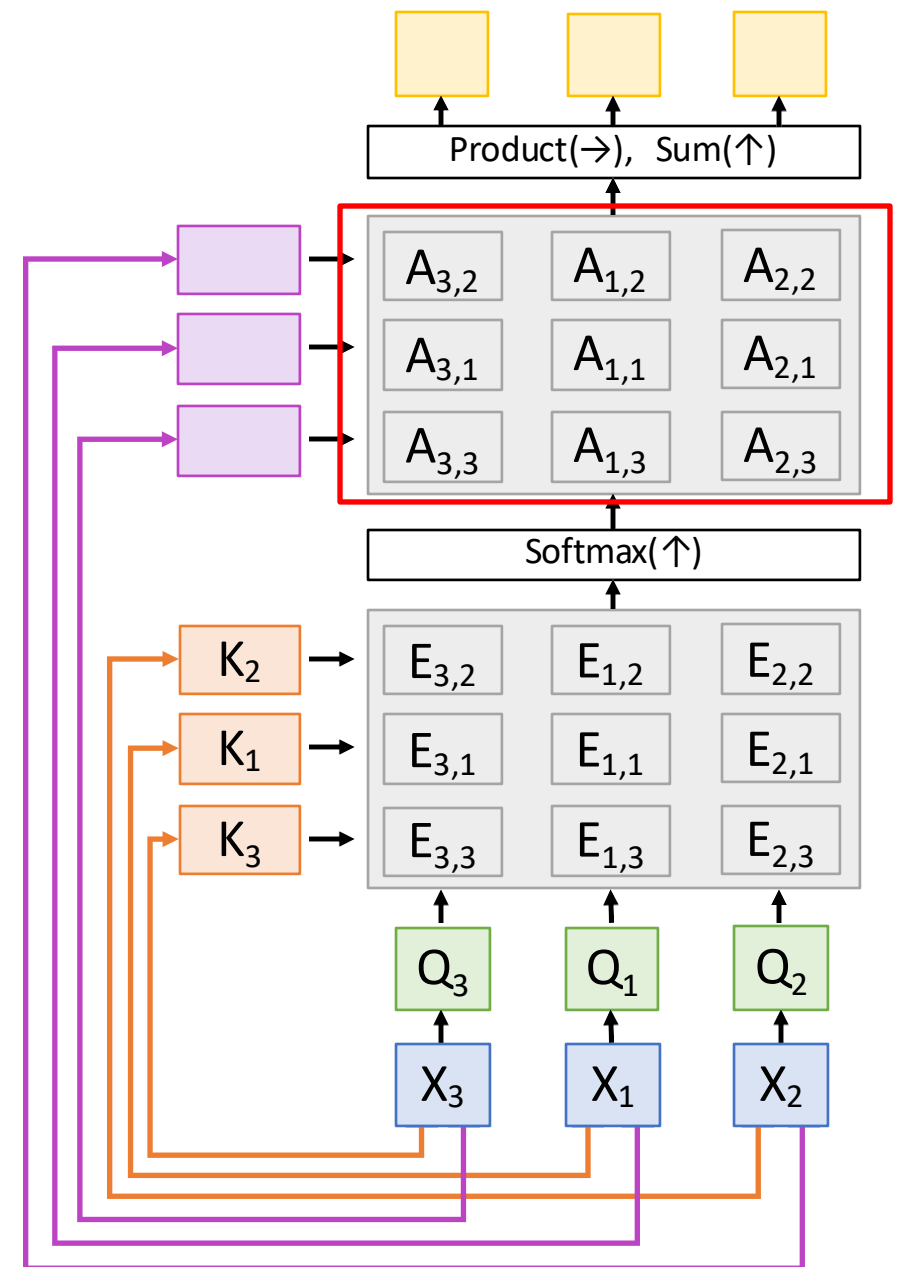
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Attention weights will be
the same, but permuted



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

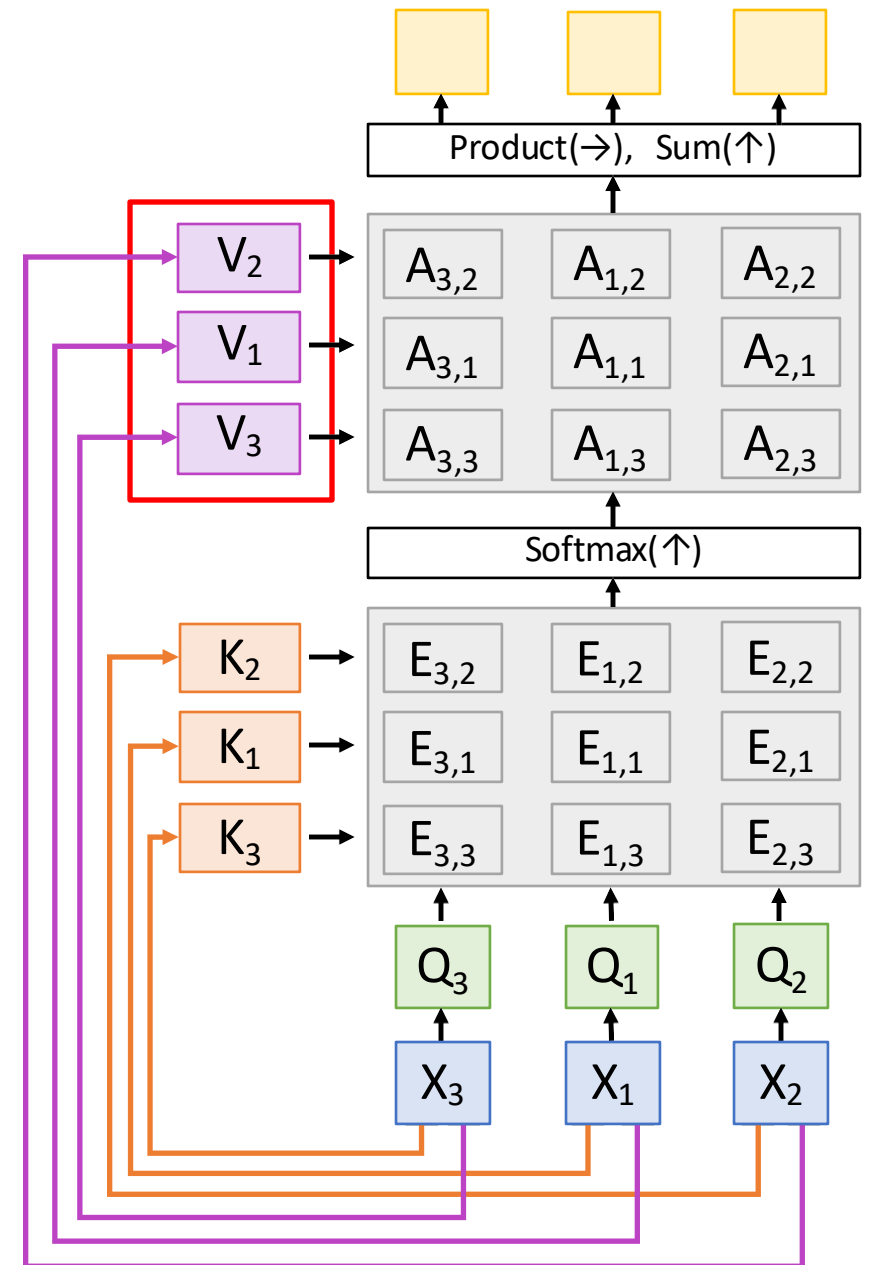
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Values will be the
same, but **permuted**



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

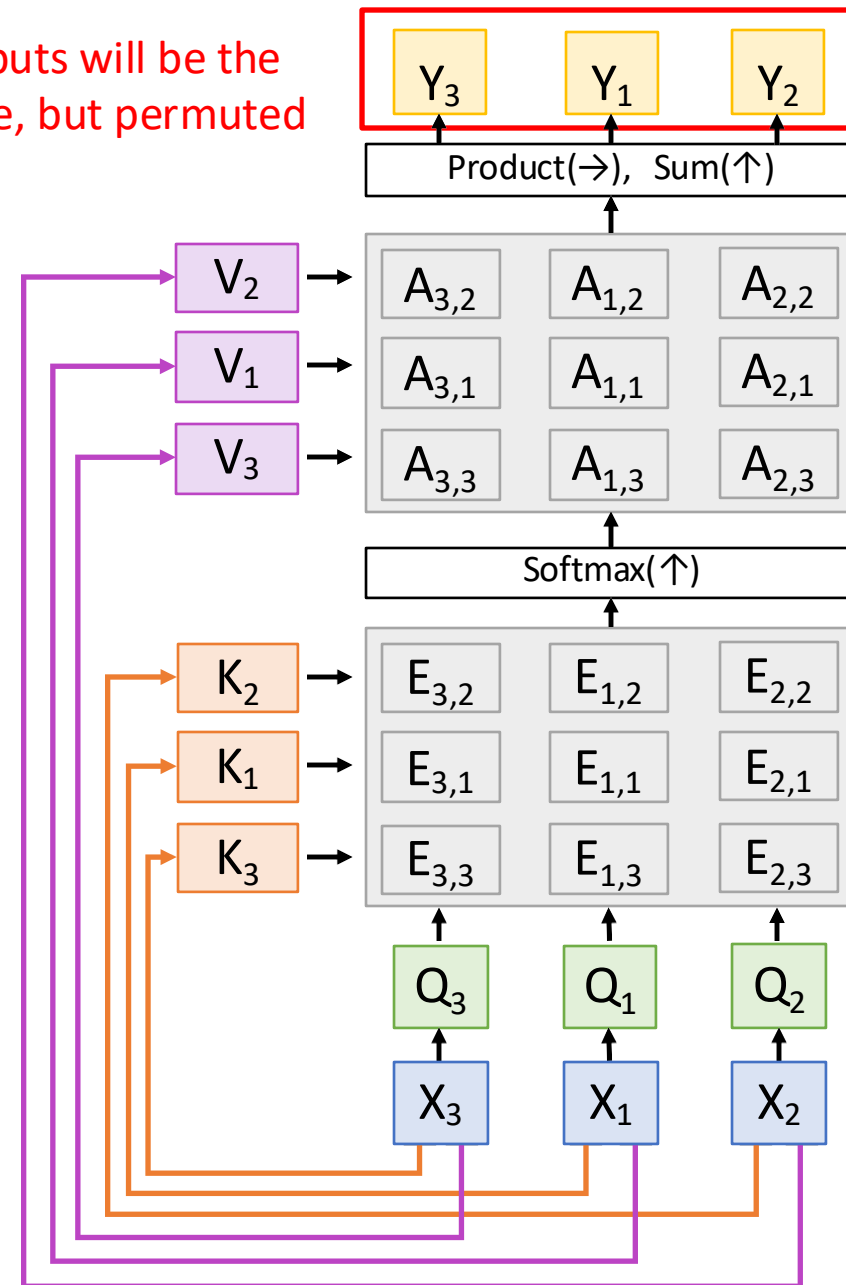
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Self-Attention layer works on **sets** of vectors

Outputs will be the same, but permuted



Positional Encoding

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

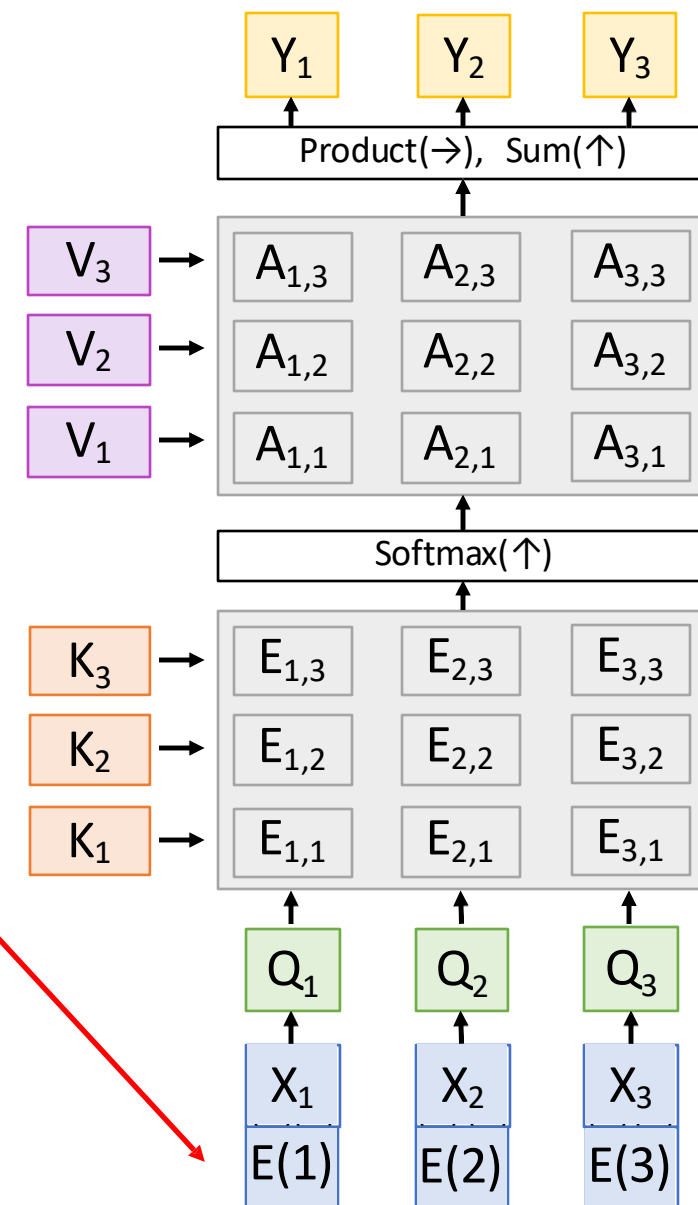
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

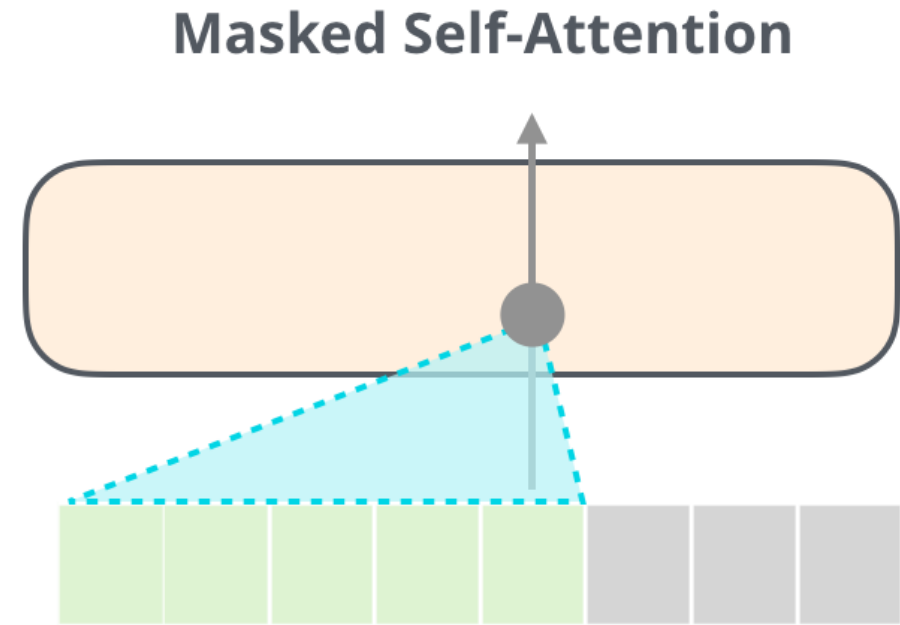
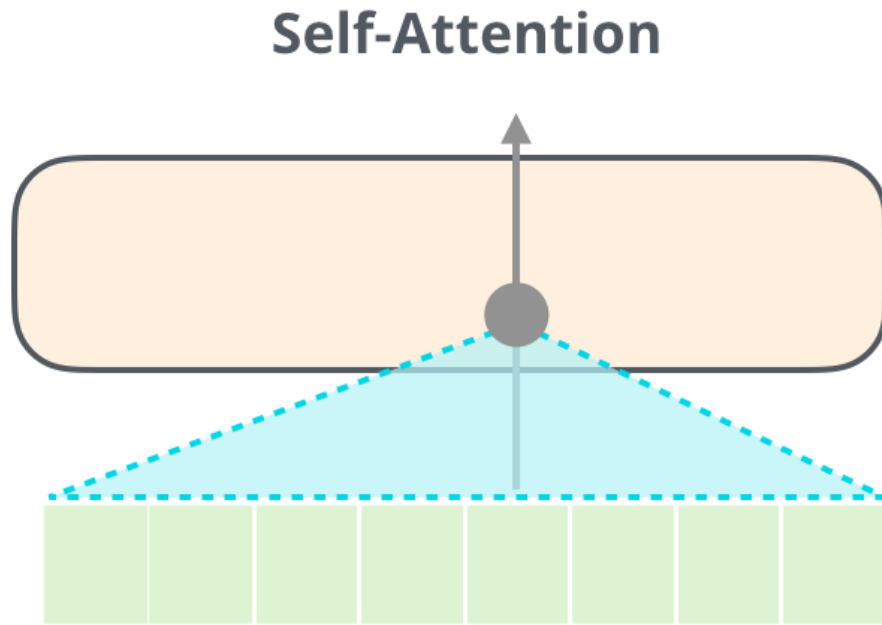
Self attention doesn't
"know" the order of the
vectors it is processing!

In order to make
processing position-
aware, concatenate input
with **positional encoding**

E can be learned lookup
table, or fixed function



Decoding: **Masked** Self-Attention



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence
Used for language modeling (predict next word)

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

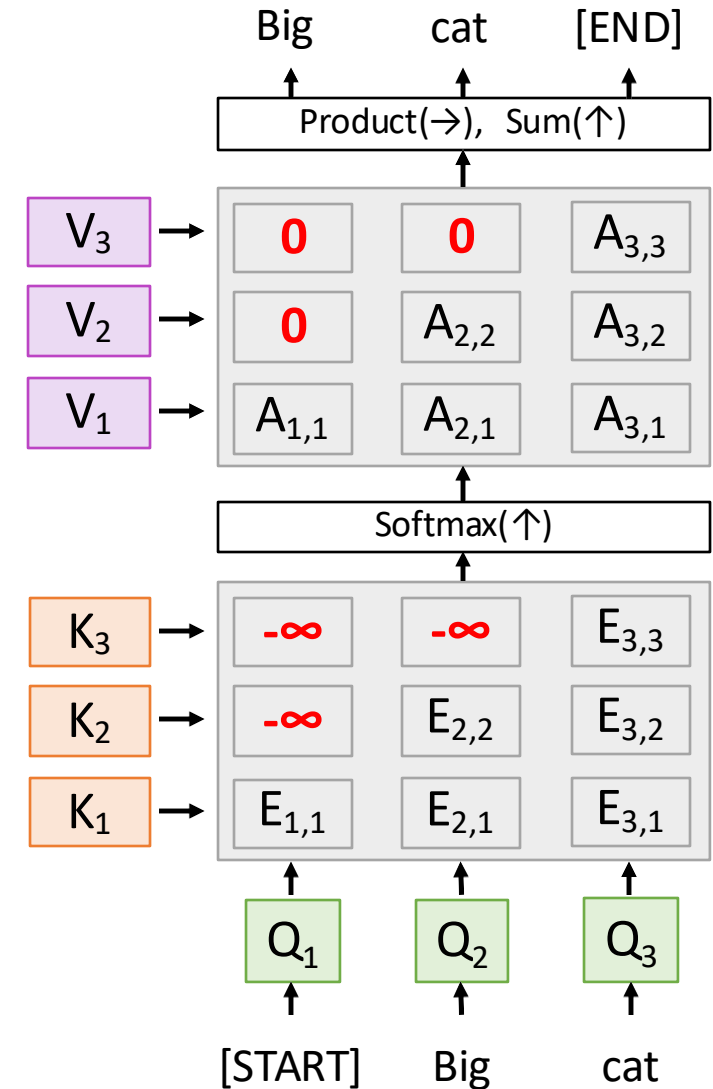
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Multihead Self-Attention Layer

Use H independent
“Attention Heads” in parallel

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_K$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_K$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

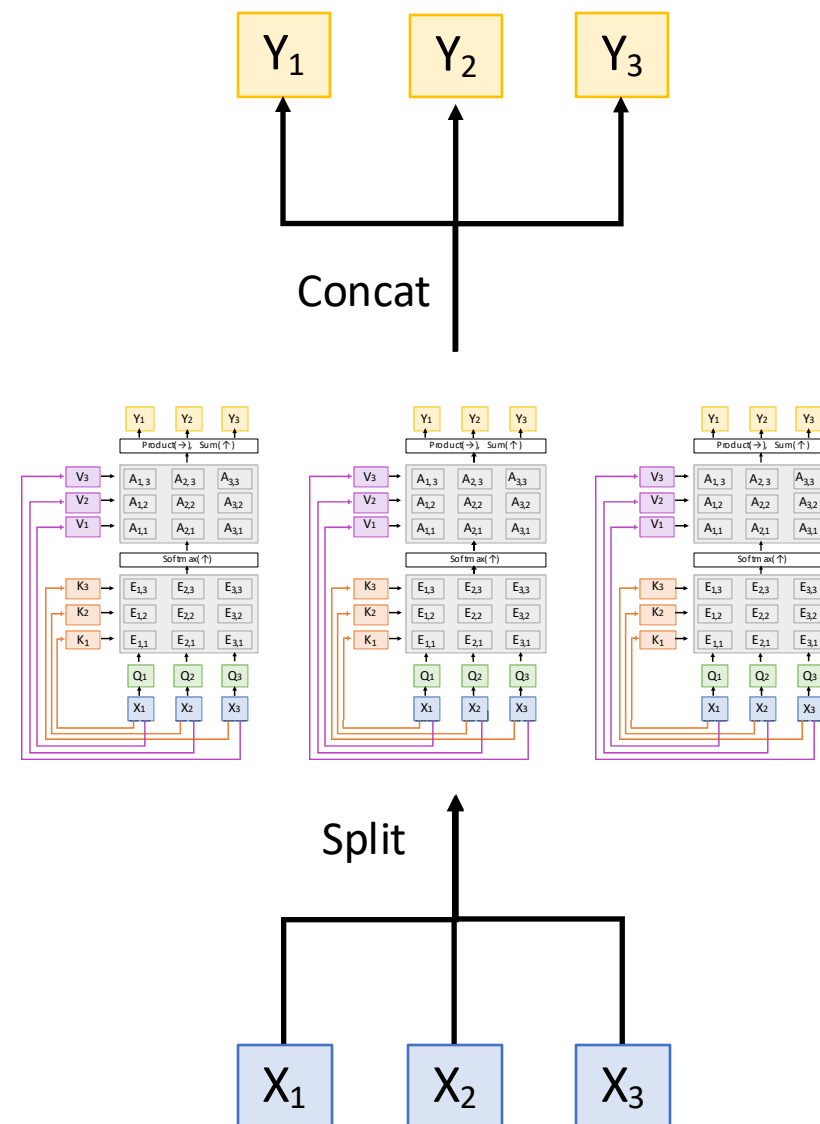
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Hyperparameters:

Query dimension D_Q

Number of heads H



Multihead Self-Attention Layer

Use H independent
“Attention Heads” in parallel

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_K$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_K$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

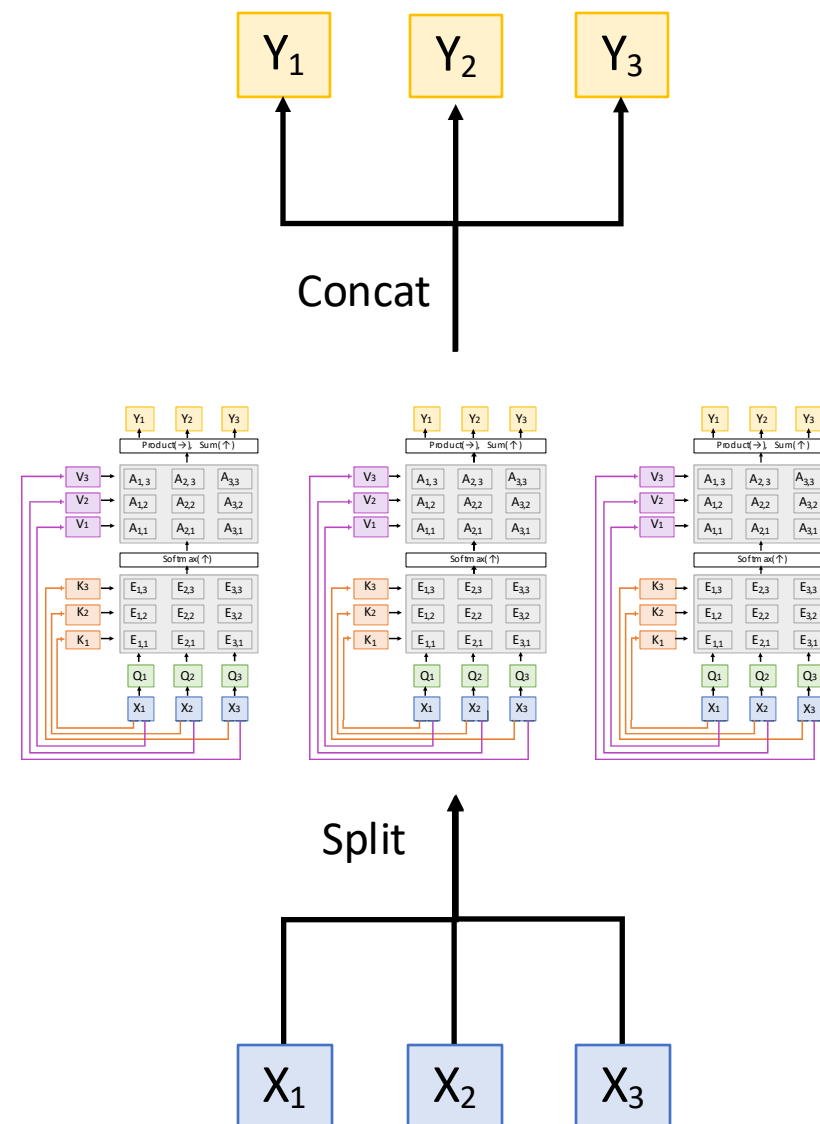
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Hyperparameters:

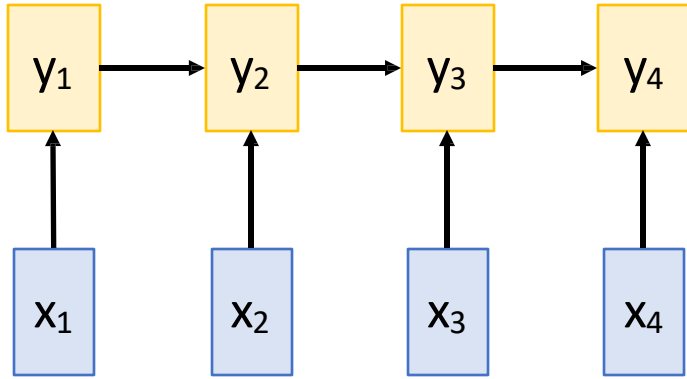
Query dimension D_Q

Number of heads H



Summary: Three Ways of Processing Sequences

Recurrent Neural Network



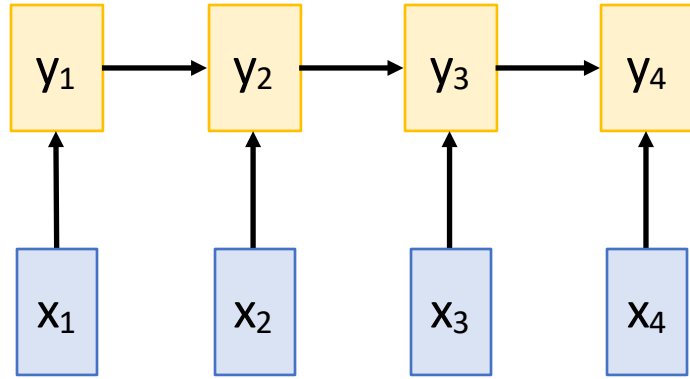
Works on **Ordered Sequences**

(+) **Good at long sequences:** After one RNN layer, h_T "sees" the whole sequence

(-) **Not parallelizable:** need to compute hidden states sequentially

Summary: Three Ways of Processing Sequences

Recurrent Neural Network

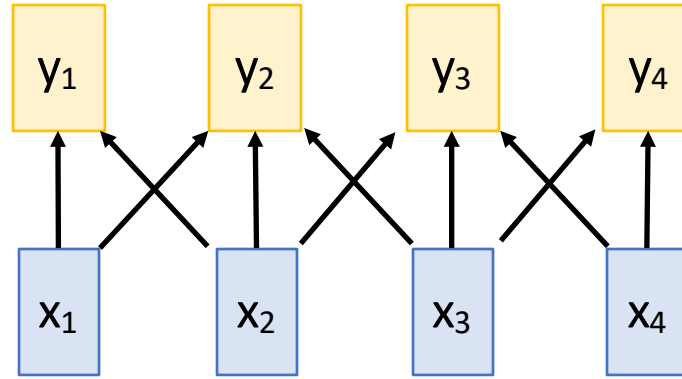


Works on **Ordered Sequences**

(+) **Good at long sequences:** After one RNN layer, h_T "sees" the whole sequence

(-) **Not parallelizable:** need to compute hidden states sequentially

1D Convolution



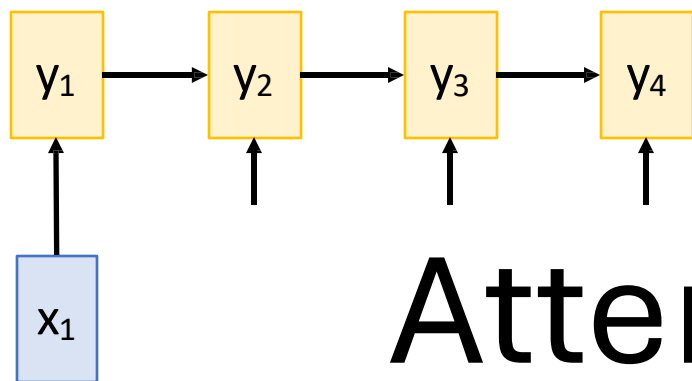
Works on **Multidimensional Grids**

(-) **Bad at long sequences:** Need to stack many conv layers for outputs to "see" the whole sequence

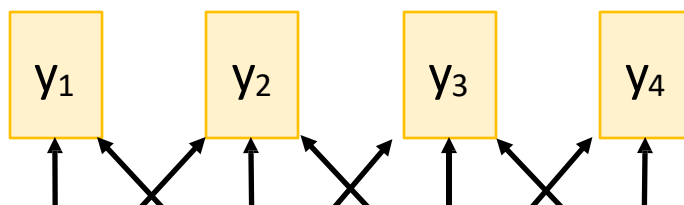
(+) **Highly parallel:** Each output can be computed in parallel

Summary: Three Ways of Processing Sequences

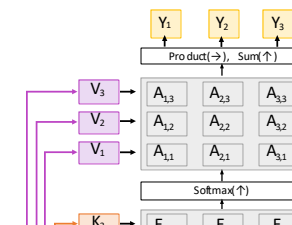
Recurrent Neural Network



1D Convolution



Self-Attention



Attention is all you need

Works on **Ordered Sequences**

(+) **Good at long sequences:** After one RNN layer, h_T "sees" the whole sequence

(-) **Not parallelizable:** need to compute hidden states sequentially

Works on **Multidimensional Grids**

(-) **Bad at long sequences:** Need to stack many conv layers for outputs to "see" the whole sequence

(+) **Highly parallel:** Each output can be computed in parallel

Works on **Sets of Vectors**

(-) **Good at long sequences:** after one self-attention layer, each output "sees" all inputs!

(+) **Highly parallel:** Each output can be computed in parallel

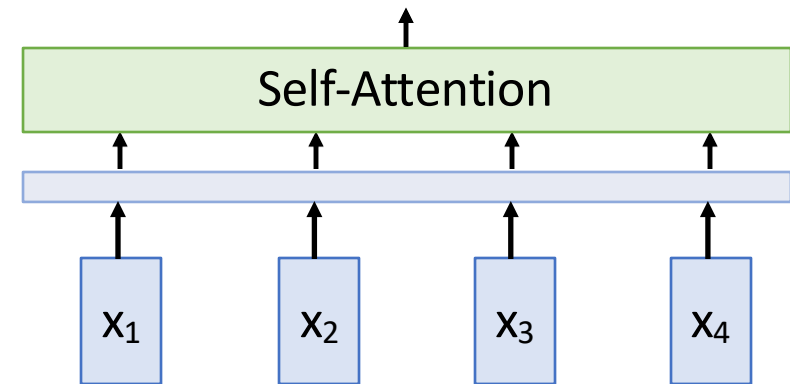
(-) **Very memory intensive**

The Transformer



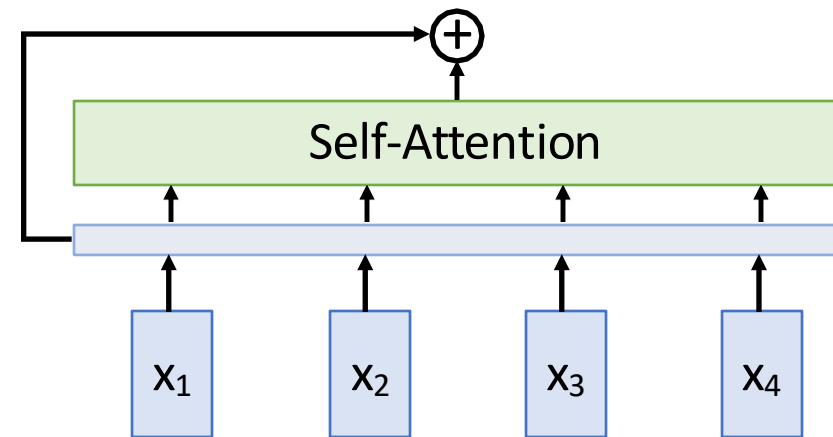
The Transformer

All vectors interact
with each other



The Transformer

Residual connection
All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

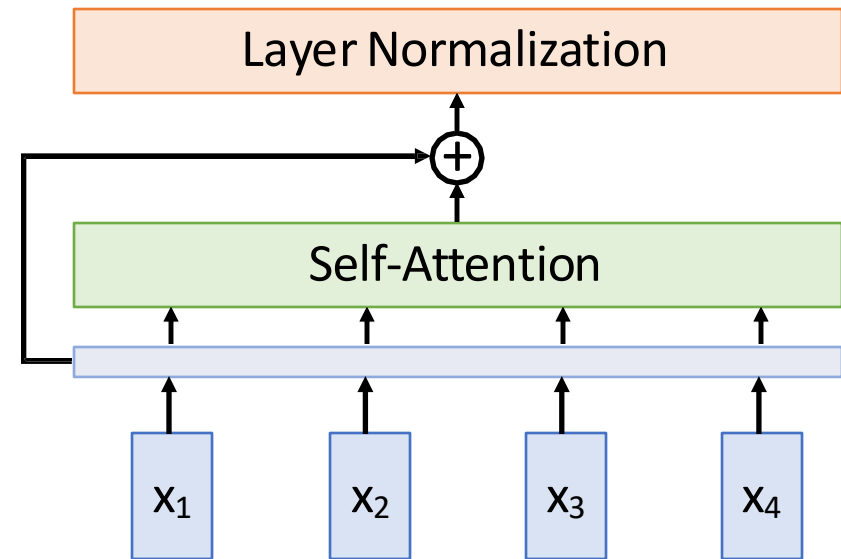
$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

Ba et al, 2016

Residual connection
All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

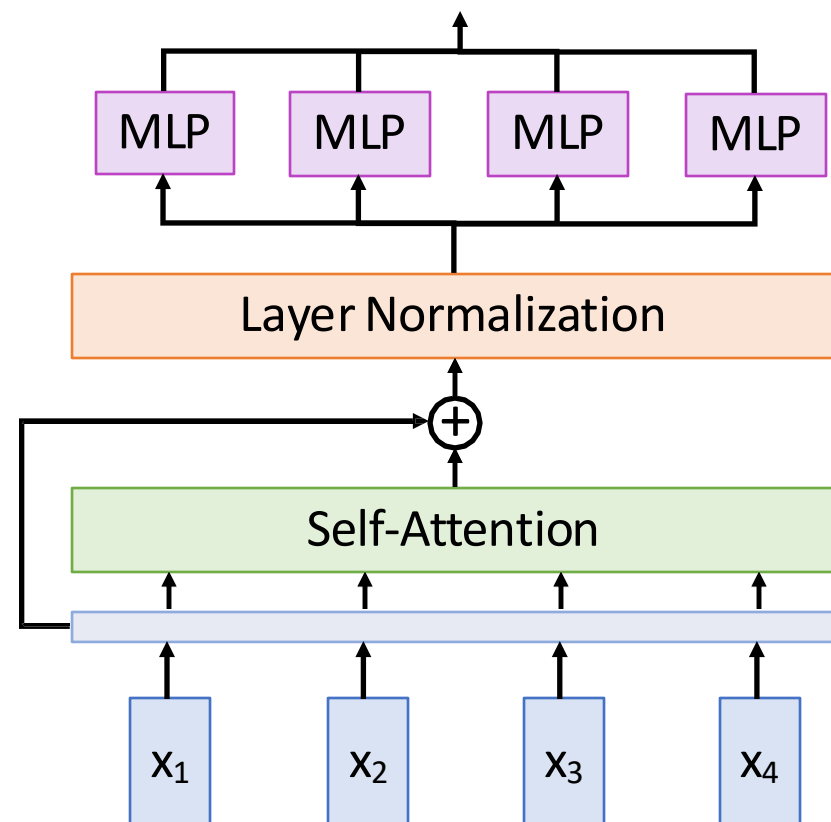
$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

Ba et al, 2016

MLP independently
on each vector

Residual connection
All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

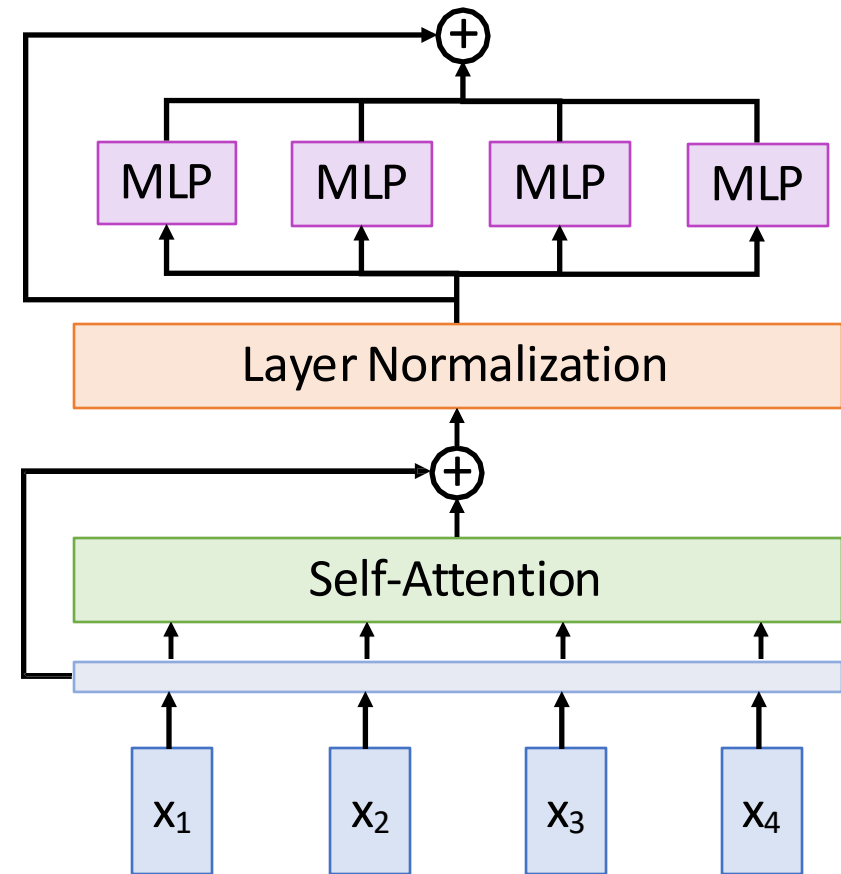
Ba et al, 2016

Residual connection

MLP independently
on each vector

Residual connection

All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

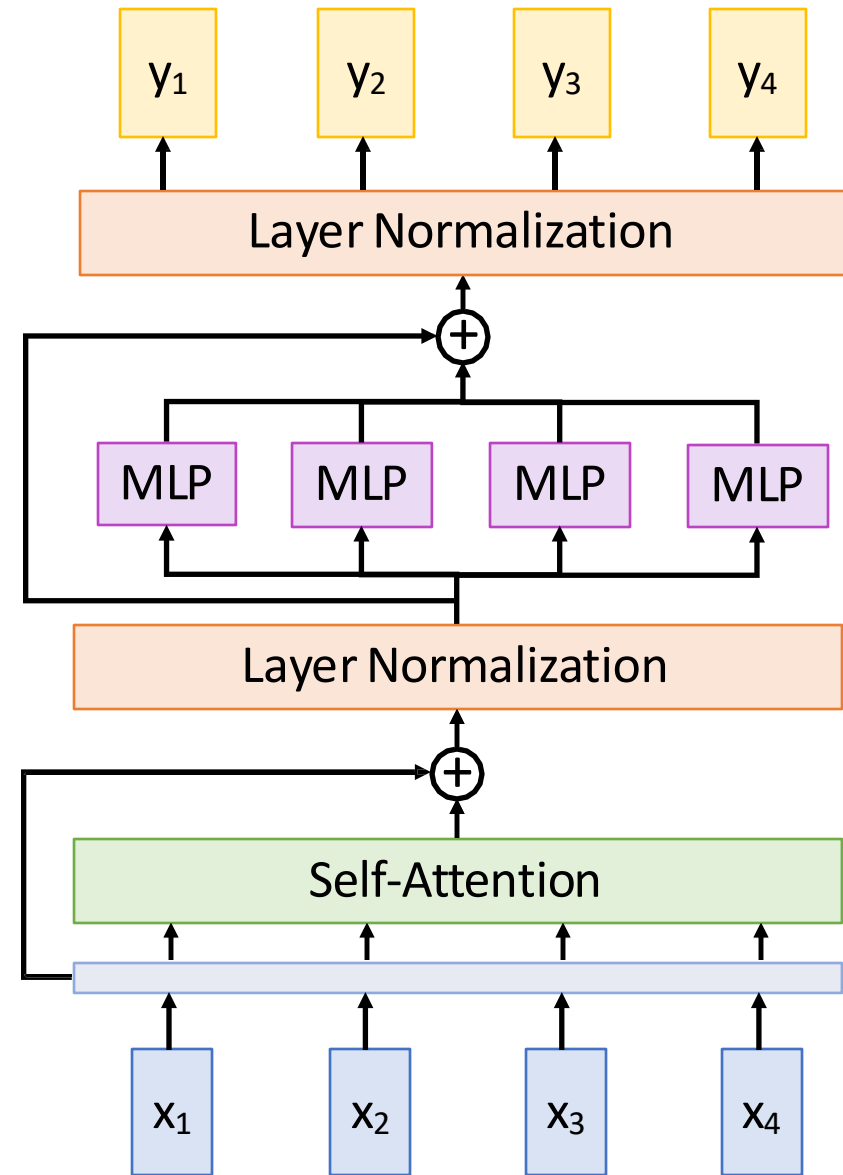
Ba et al, 2016

Residual connection

MLP independently
on each vector

Residual connection

All vectors interact
with each other



The Transformer

Transformer Block:

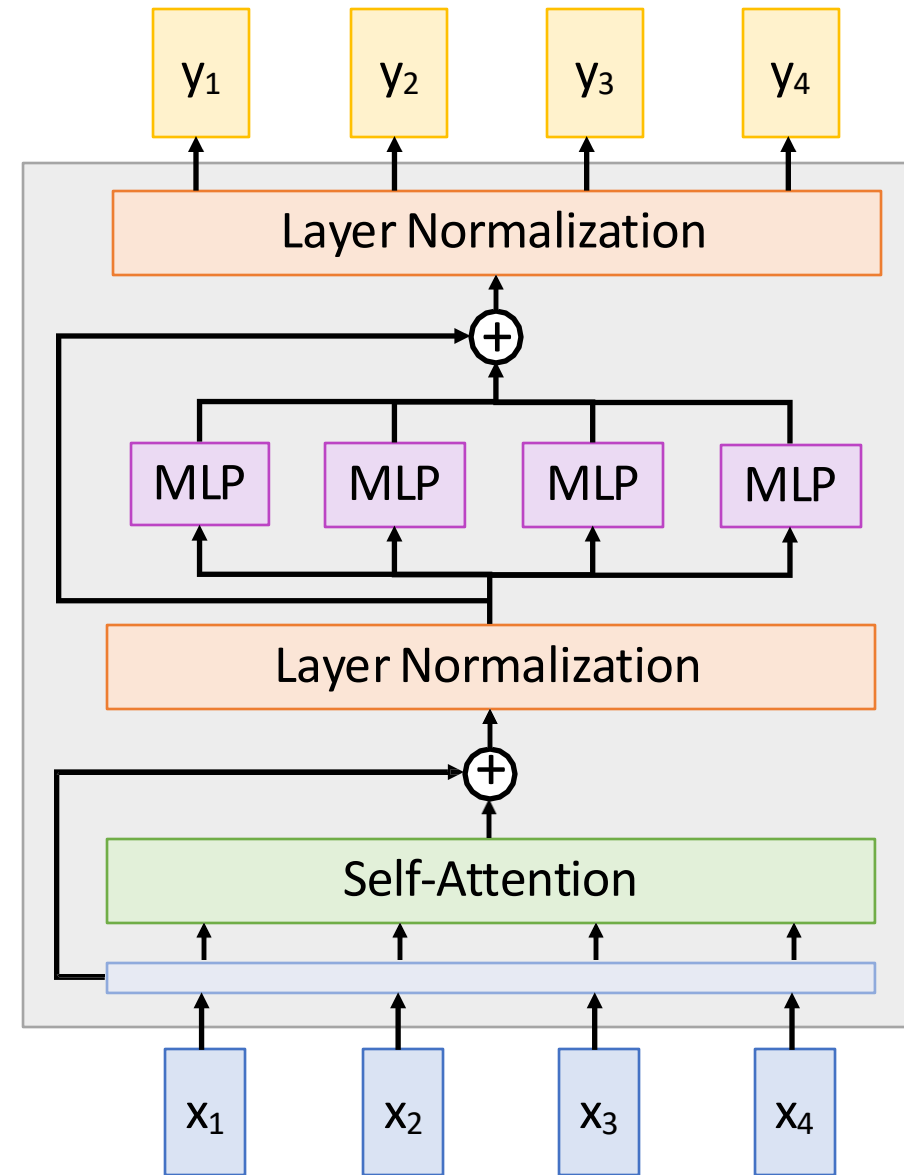
Input: Set of vectors x

Output: Set of vectors y

Self-attention is the only interaction between vectors!

Layer norm and MLP work independently per vector

Highly scalable, highly parallelizable



The Transformer

Transformer Block:

Input: Set of vectors x

Output: Set of vectors y

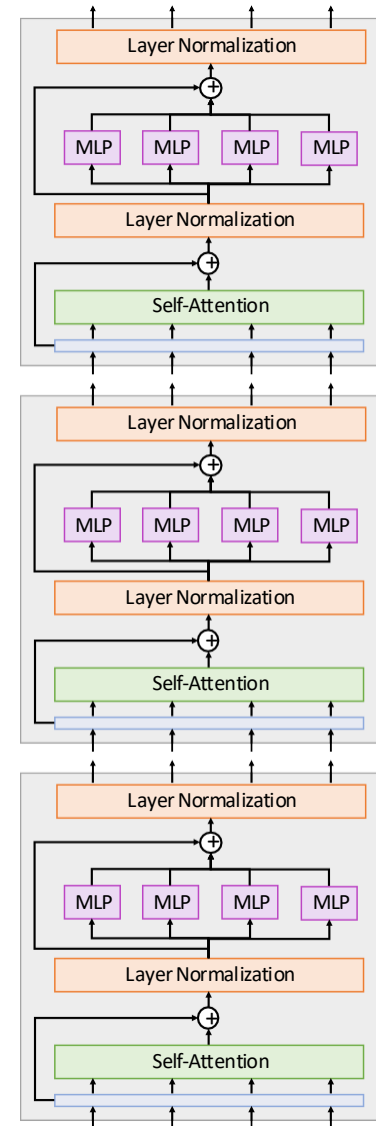
Self-attention is the only
interaction between vectors!

Layer norm and MLP work
independently per vector

Highly scalable, highly
parallelizable

A **Transformer** is a sequence
of transformer blocks

Vaswani et al:
12 blocks, $D_Q=512$, 6 heads



The Transformer: Transfer Learning

“ImageNet Moment for Natural Language Processing”

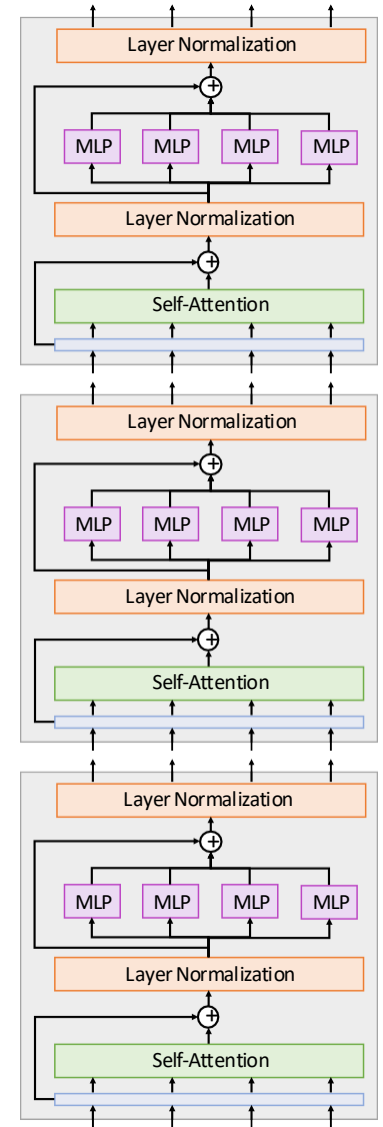
Pretraining:

Download a lot of text from the internet

Train a giant Transformer model for language modeling

Finetuning:

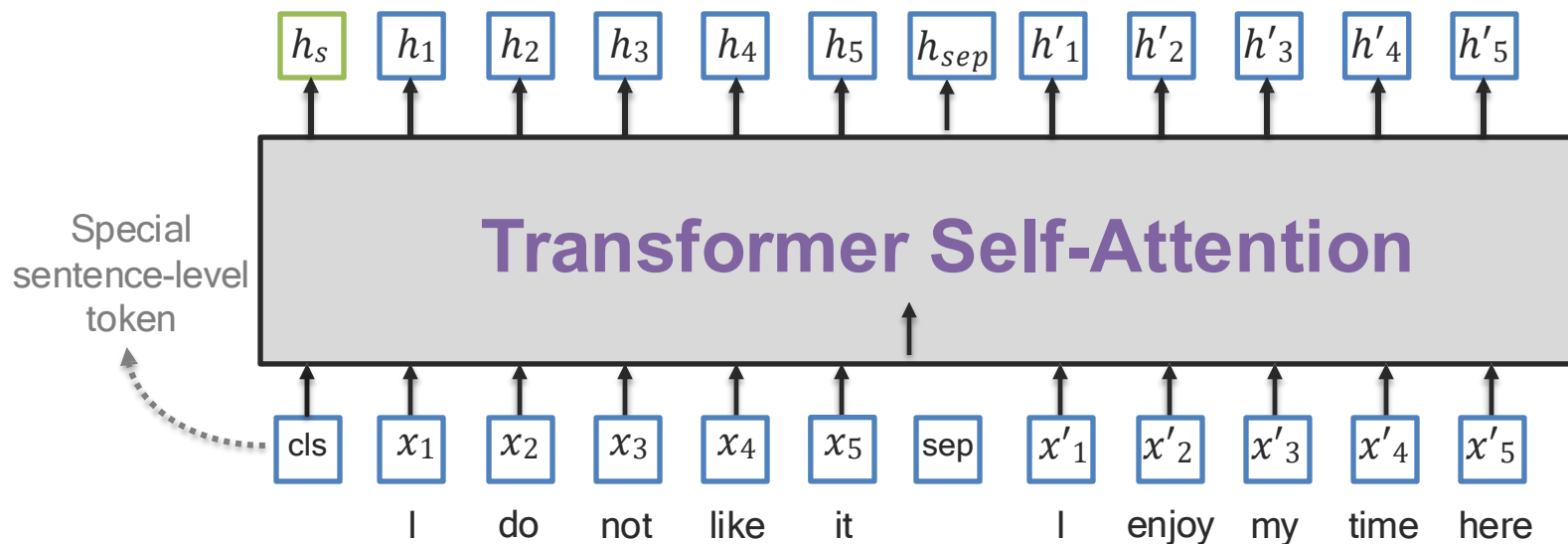
Fine-tune the Transformer on your own NLP task



BERT: Bidirectional Encoder Representations

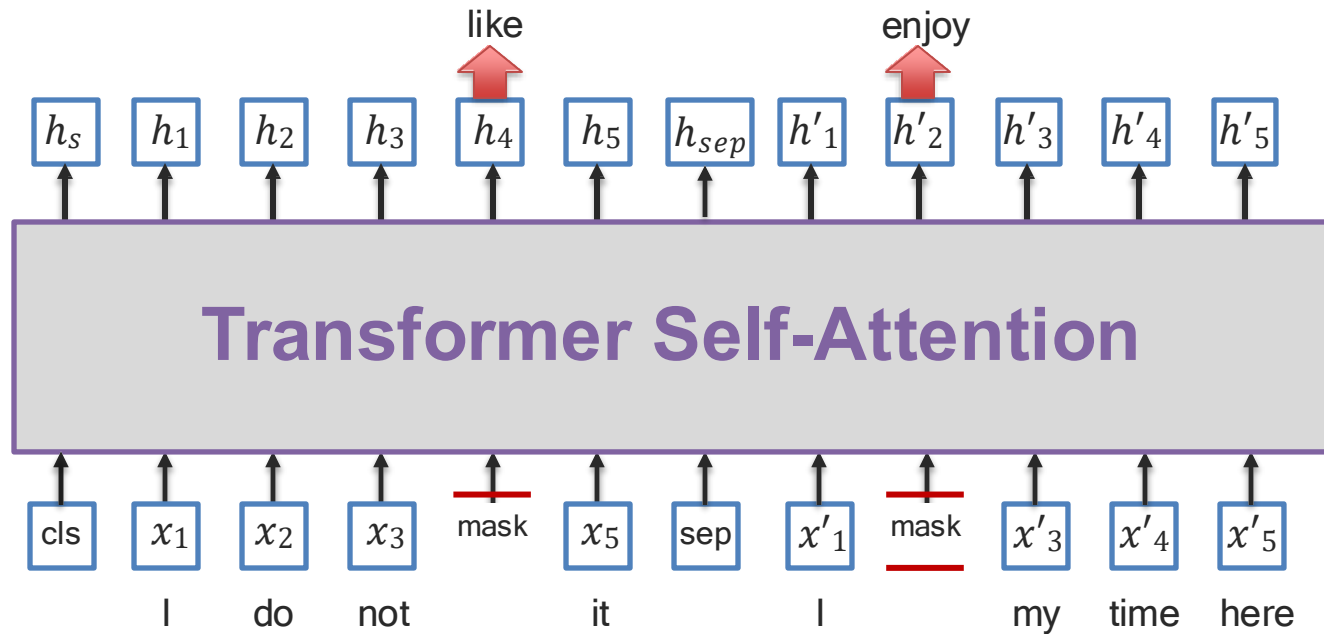
Advantages:

- 1 Jointly learn representation for token-level and sentence level
- 2 Same network architecture for pre-training and fine-tuning
- 3 Can be used learn relationship between sentences
- 4 Models bidirectional and long-range interactions between tokens



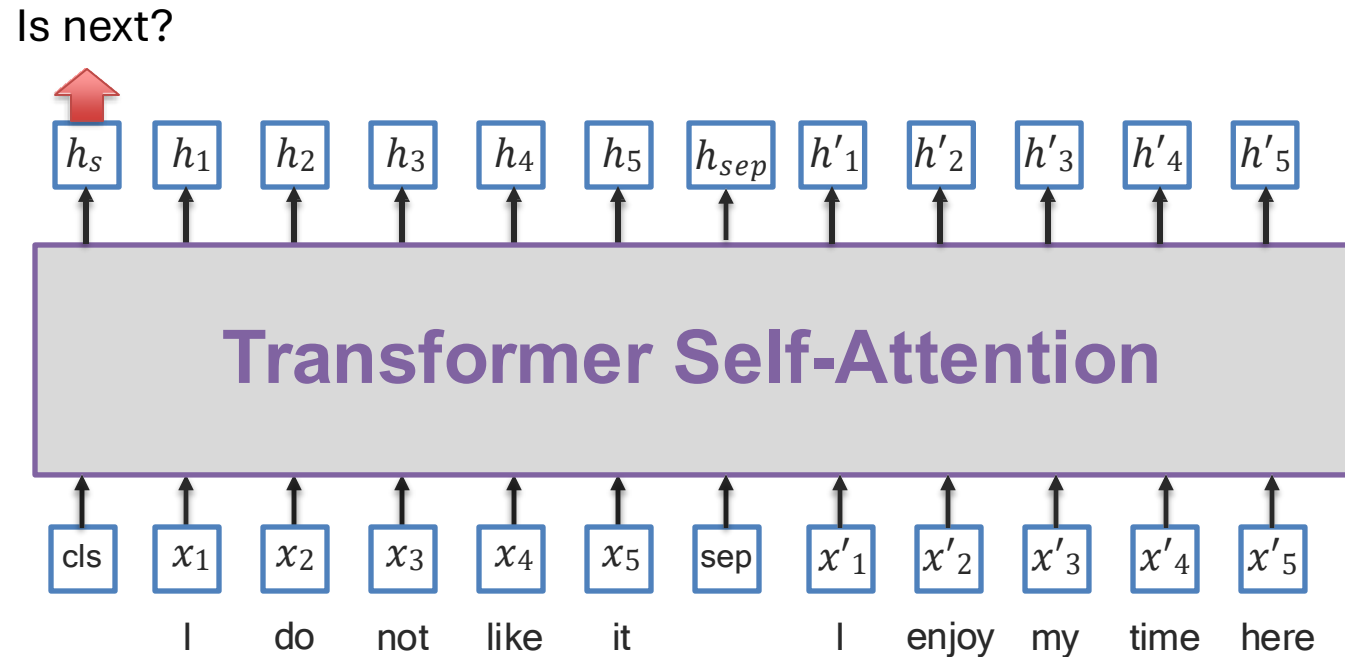
Self-supervised Pre-training

- **Masked Language Model:** Randomly mask input tokens and then try to predict them



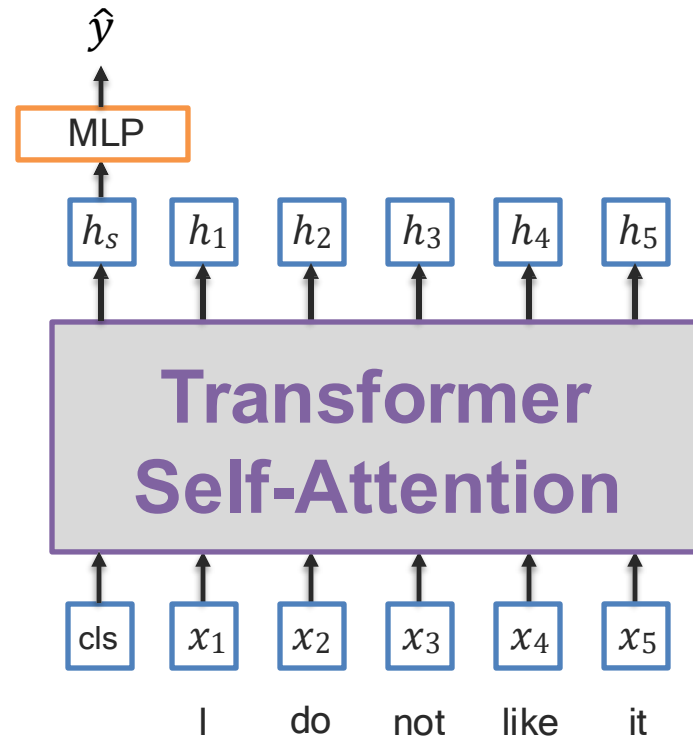
Self-supervised Pre-training

- **Next Sentence Prediction:** Given two sentences, predict if this is the next one or not



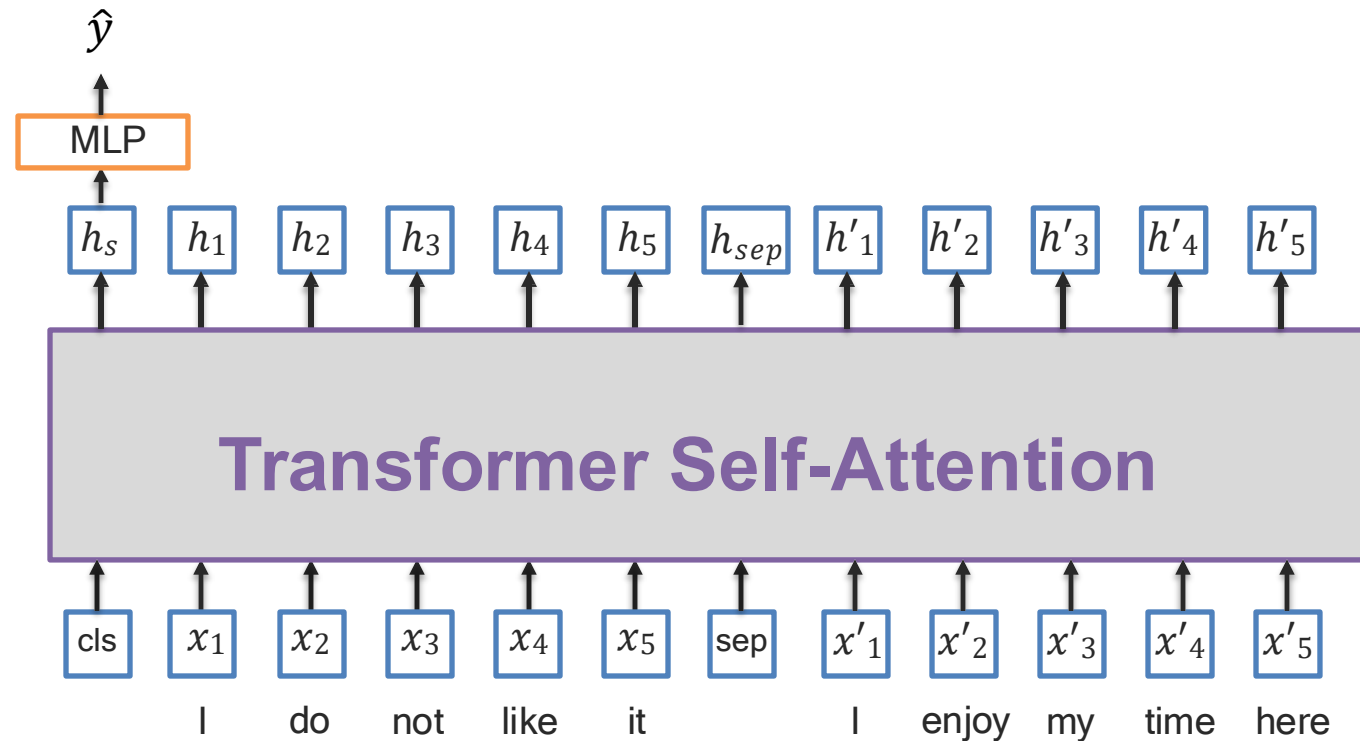
Fine-tuning

- Sentence-level classification



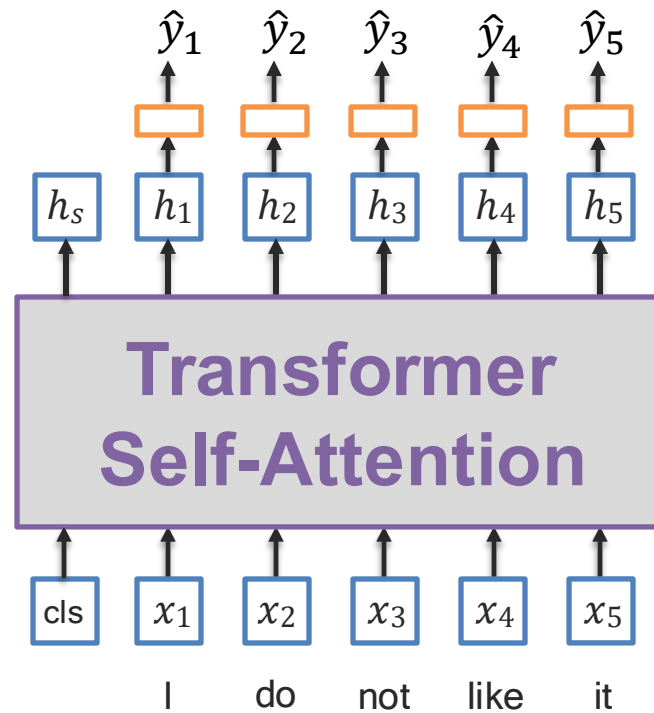
Fine-tuning

- More than one sentence

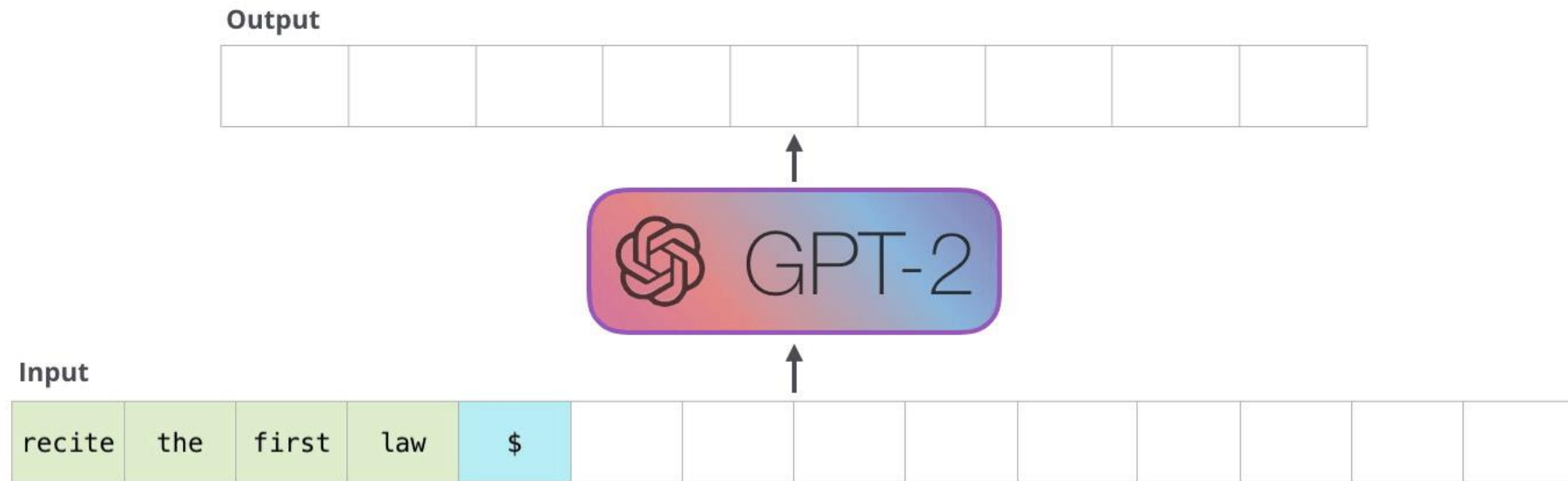


Fine-tuning

- Token-level classification



GPT-2 showed - autoregressive decoding is all you need



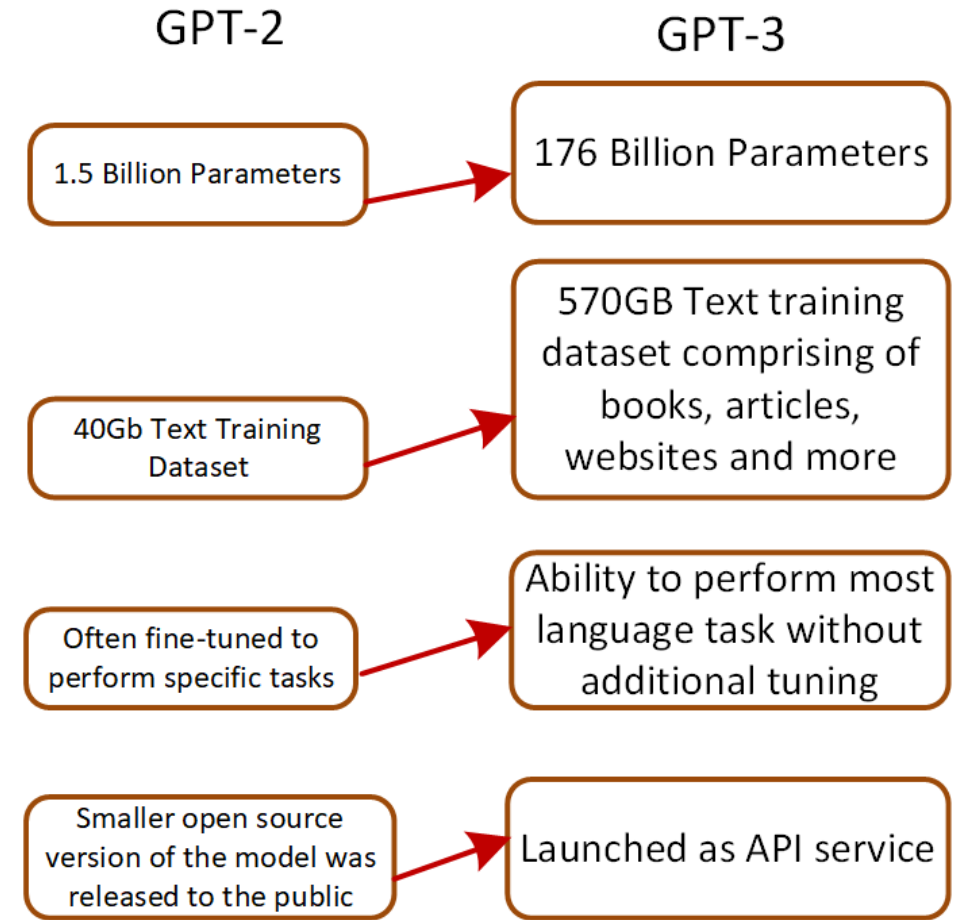
Scaling up Transformers

~\$430,000 on Amazon AWS!

Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	12	768	?	117M	40 GB	
GPT-2	24	1024	?	345M	40 GB	
GPT-2	36	1280	?	762M	40 GB	
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	40	1536	16	1.2B	174 GB	64x V100 GPU
Megatron-LM	54	1920	20	2.5B	174 GB	128x V100 GPU
Megatron-LM	64	2304	24	4.2B	174 GB	256x V100 GPU (10 days)
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)

GPT-3; Easy to scale

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128

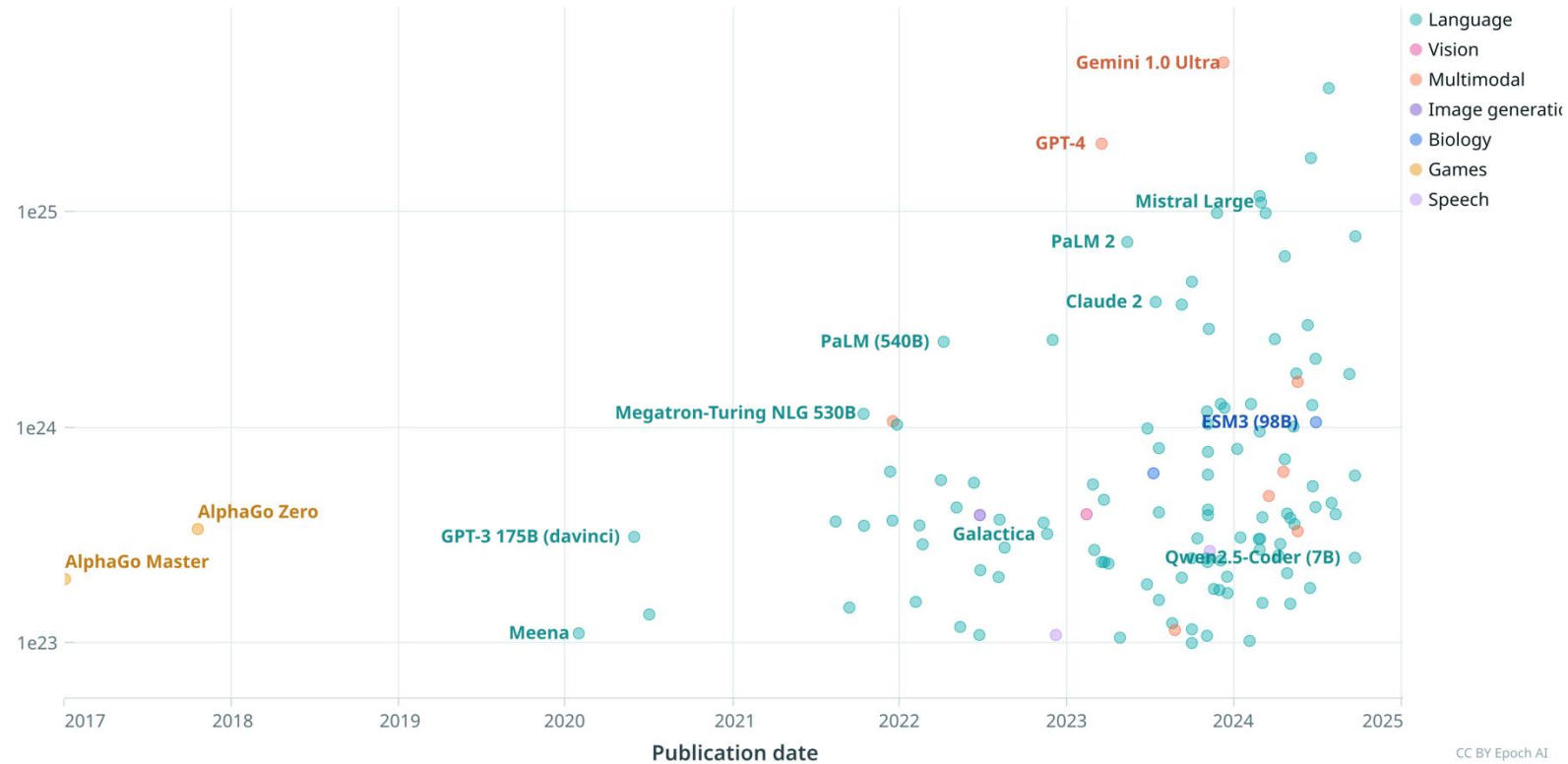


LLMs Moore's Law

Large-Scale AI Models

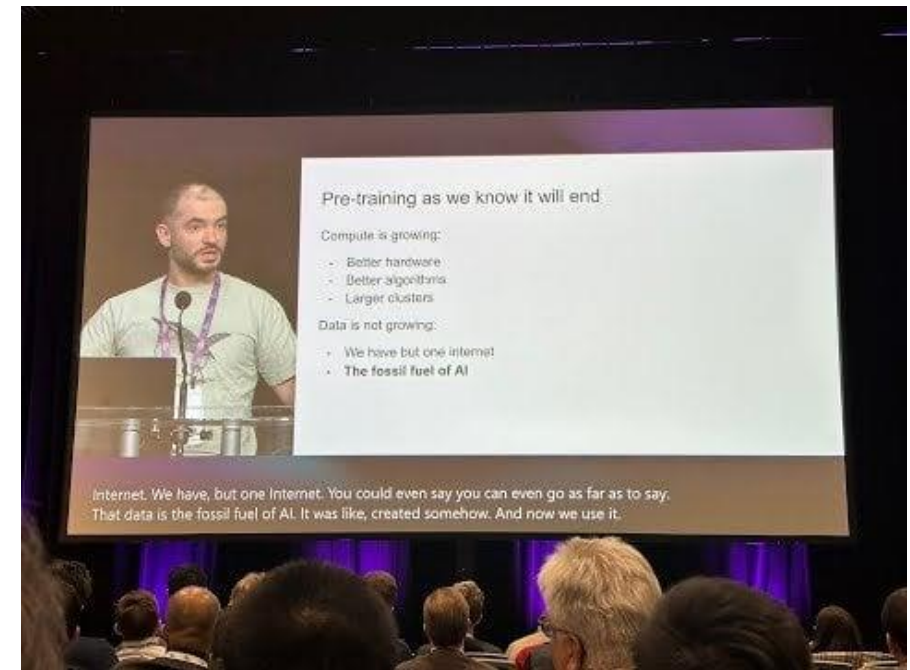
EPOCH AI

Training compute (FLOP)



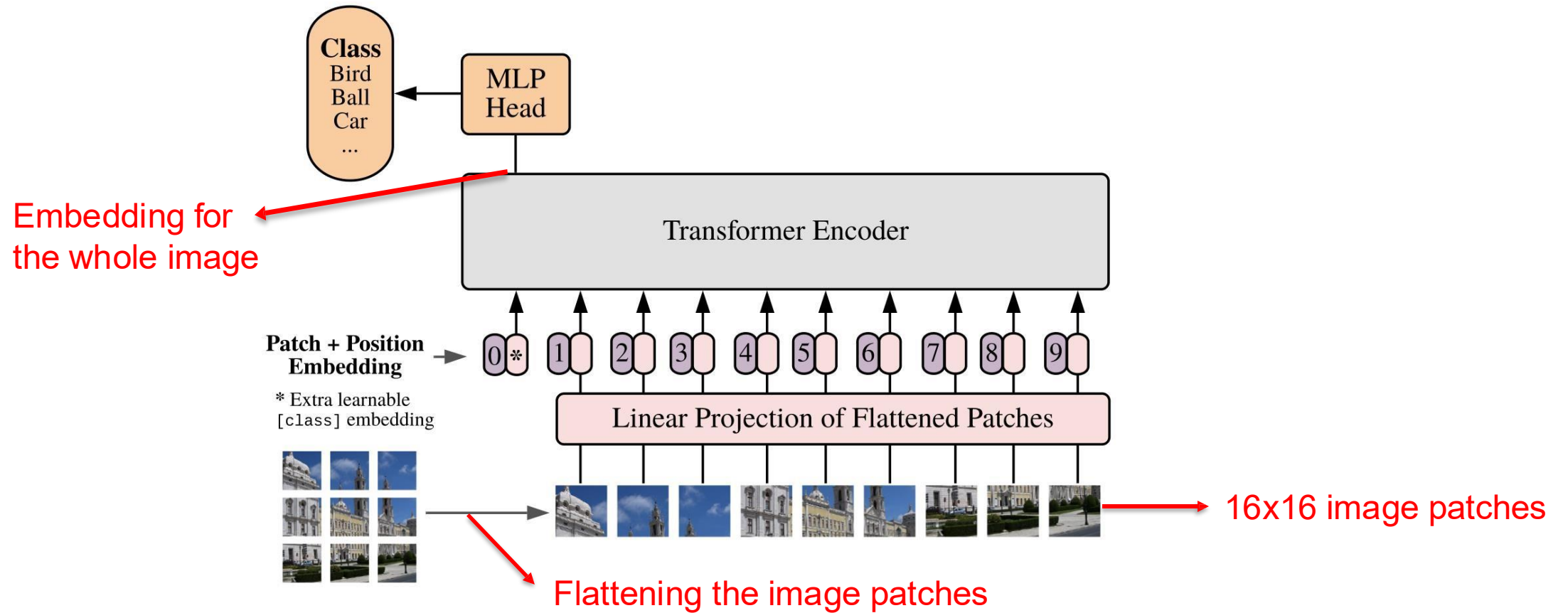
Can pre-training continue?

- Compute is growing:
 - Better hardware
 - Better algorithms
 - Larger clusters
- Data is not growing:
 - We have but one internet
 - The fossil fuel of AI
- **RL Post-training?**
 - **Maybe; we'll discuss**



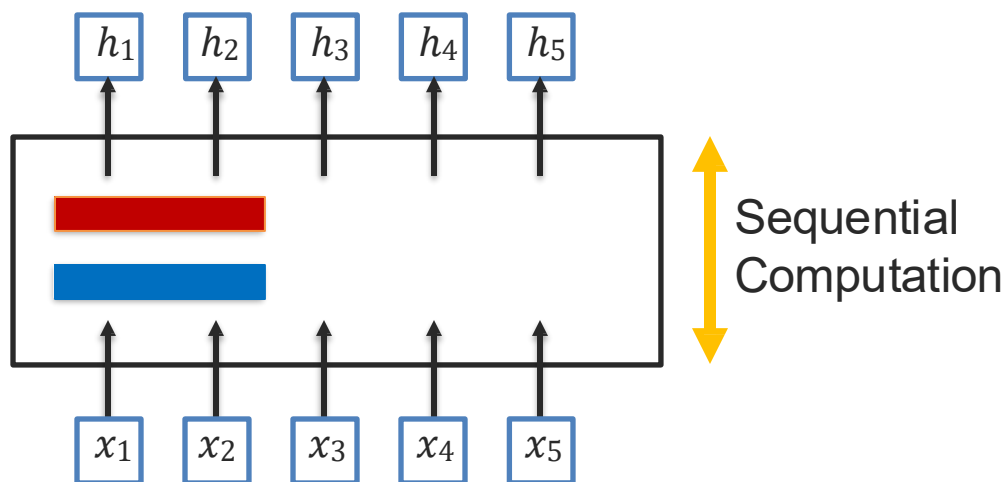
Ilya Sutskever at Neurips'24

Vision Transformer



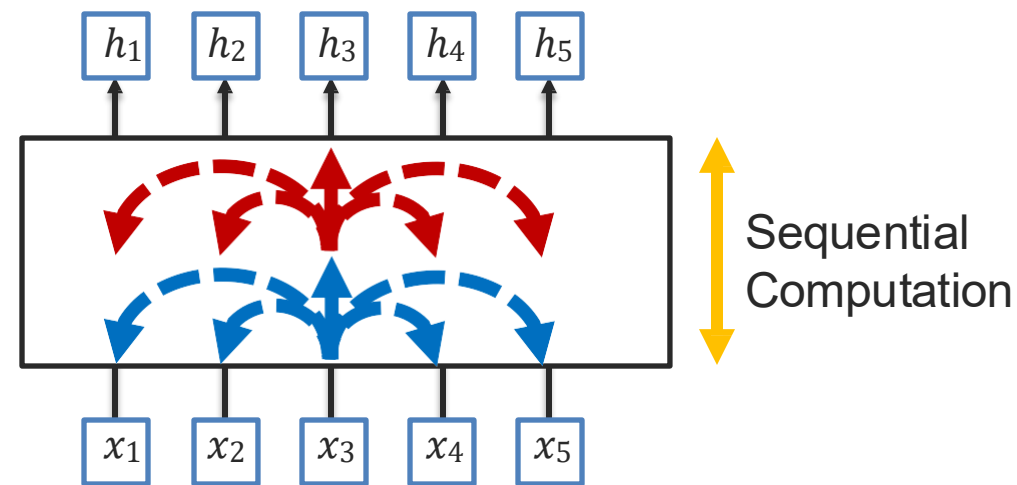
CNNs vs Transformers

Convolutions



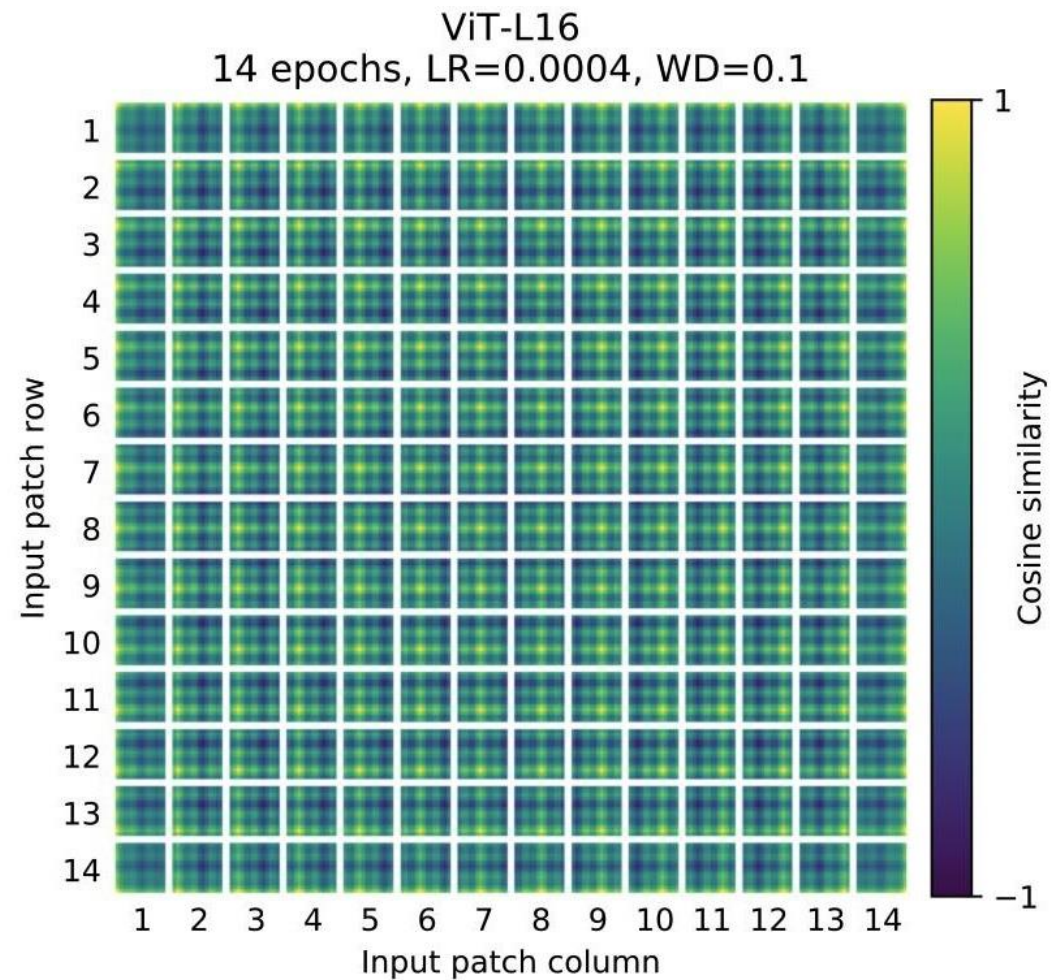
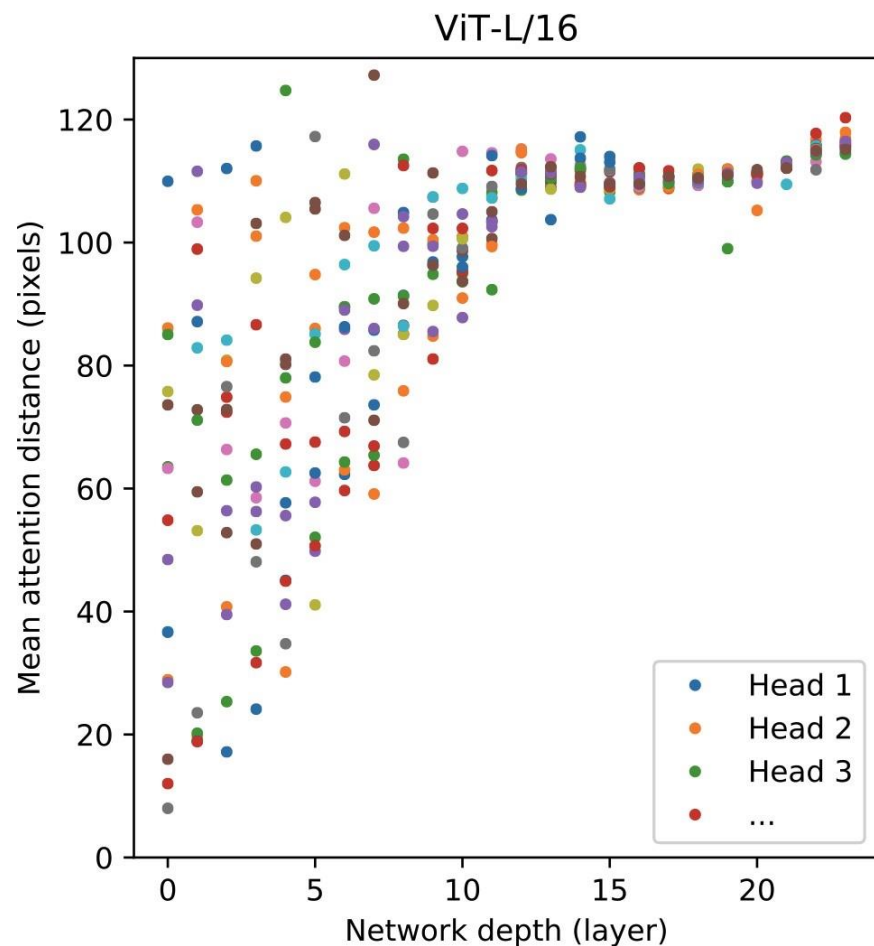
Modeling long-range dependencies requires many layers.

Self-attention



No inductive bias toward locality

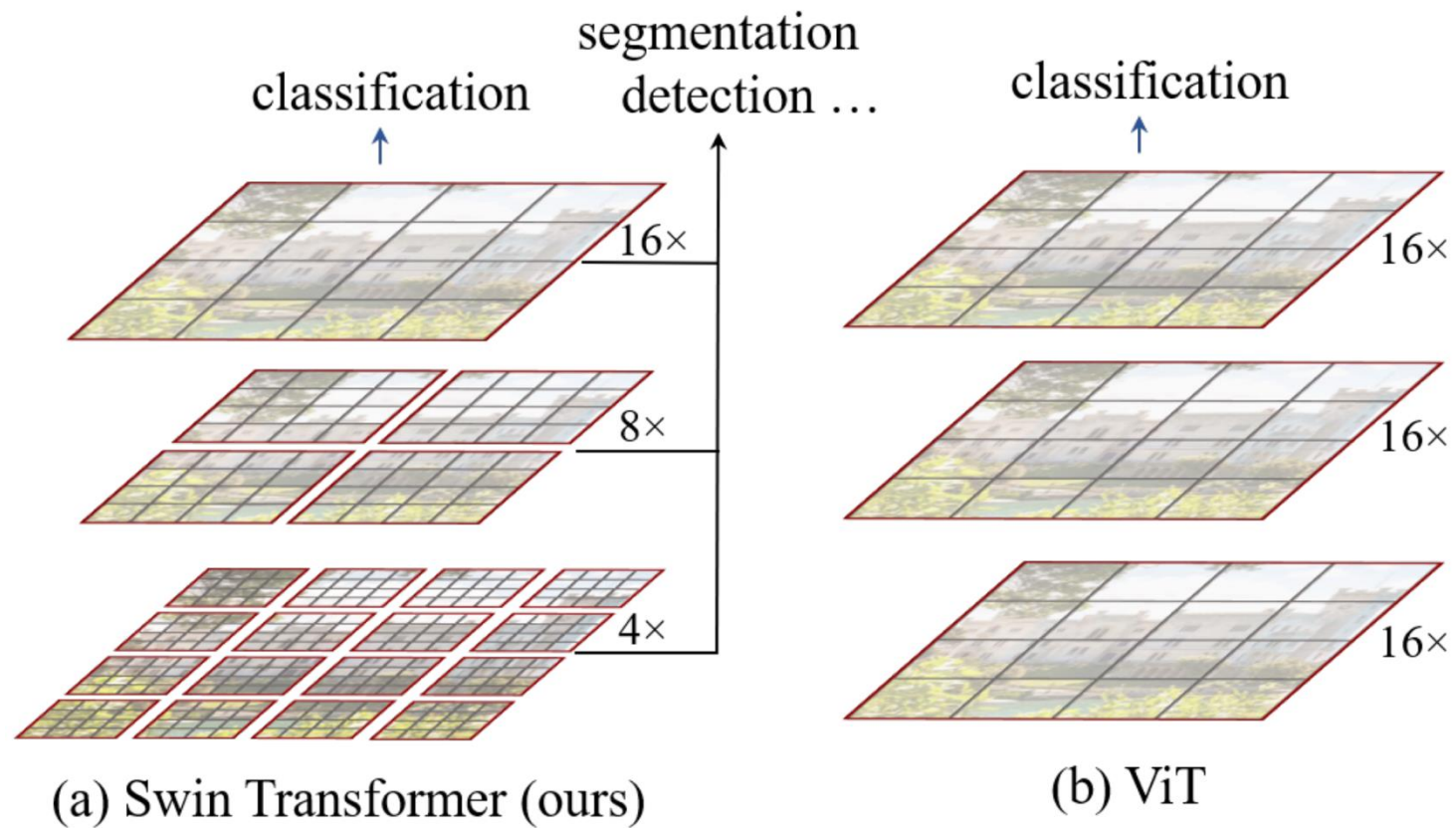
ViT do learn locality bias



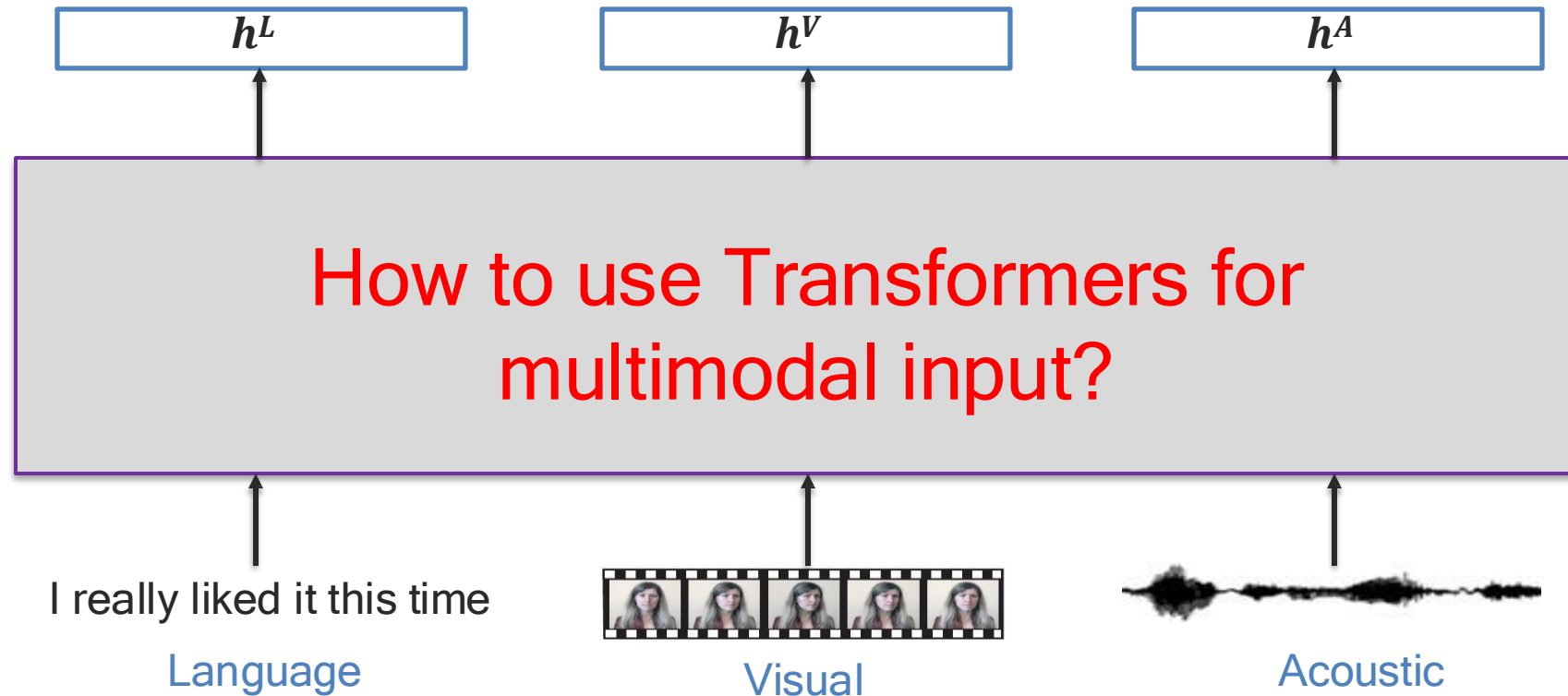
Which one is better? Approximately the same.

Model	Epochs	ImageNet	ImageNet ReaL	CIFAR-10	CIFAR-100	Pets	Flowers	exaFLOPs
ViT-B/32	7	80.73	86.27	98.61	90.49	93.40	99.27	164
ViT-B/16	7	84.15	88.85	99.00	91.87	95.80	99.56	743
ViT-L/32	7	84.37	88.28	99.19	92.52	95.83	99.45	574
ViT-L/16	7	86.30	89.43	99.38	93.46	96.81	99.66	2586
ViT-L/16	14	87.12	89.99	99.38	94.04	97.11	99.56	5172
ViT-H/14	14	88.08	90.36	99.50	94.71	97.11	99.71	12826
ResNet50x1	7	77.54	84.56	97.67	86.07	91.11	94.26	150
ResNet50x2	7	82.12	87.94	98.29	89.20	93.43	97.02	592
ResNet101x1	7	80.67	87.07	98.48	89.17	94.08	95.95	285
ResNet152x1	7	81.88	87.96	98.82	90.22	94.17	96.94	427
ResNet152x2	7	84.97	89.69	99.06	92.05	95.37	98.62	1681
ResNet152x2	14	85.56	89.89	99.24	91.92	95.75	98.75	3362
ResNet200x3	14	87.22	90.15	99.34	93.53	96.32	99.04	10212
R50x1+ViT-B/32	7	84.90	89.15	99.01	92.24	95.75	99.46	315
R50x1+ViT-B/16	7	85.58	89.65	99.14	92.63	96.65	99.40	855
R50x1+ViT-L/32	7	85.68	89.04	99.24	92.93	96.97	99.43	725
R50x1+ViT-L/16	7	86.60	89.72	99.18	93.64	97.03	99.40	2704
R50x1+ViT-L/16	14	87.12	89.76	99.31	93.89	97.36	99.11	5165

Hierarchical Vision Transformer



Multimodal Transformers

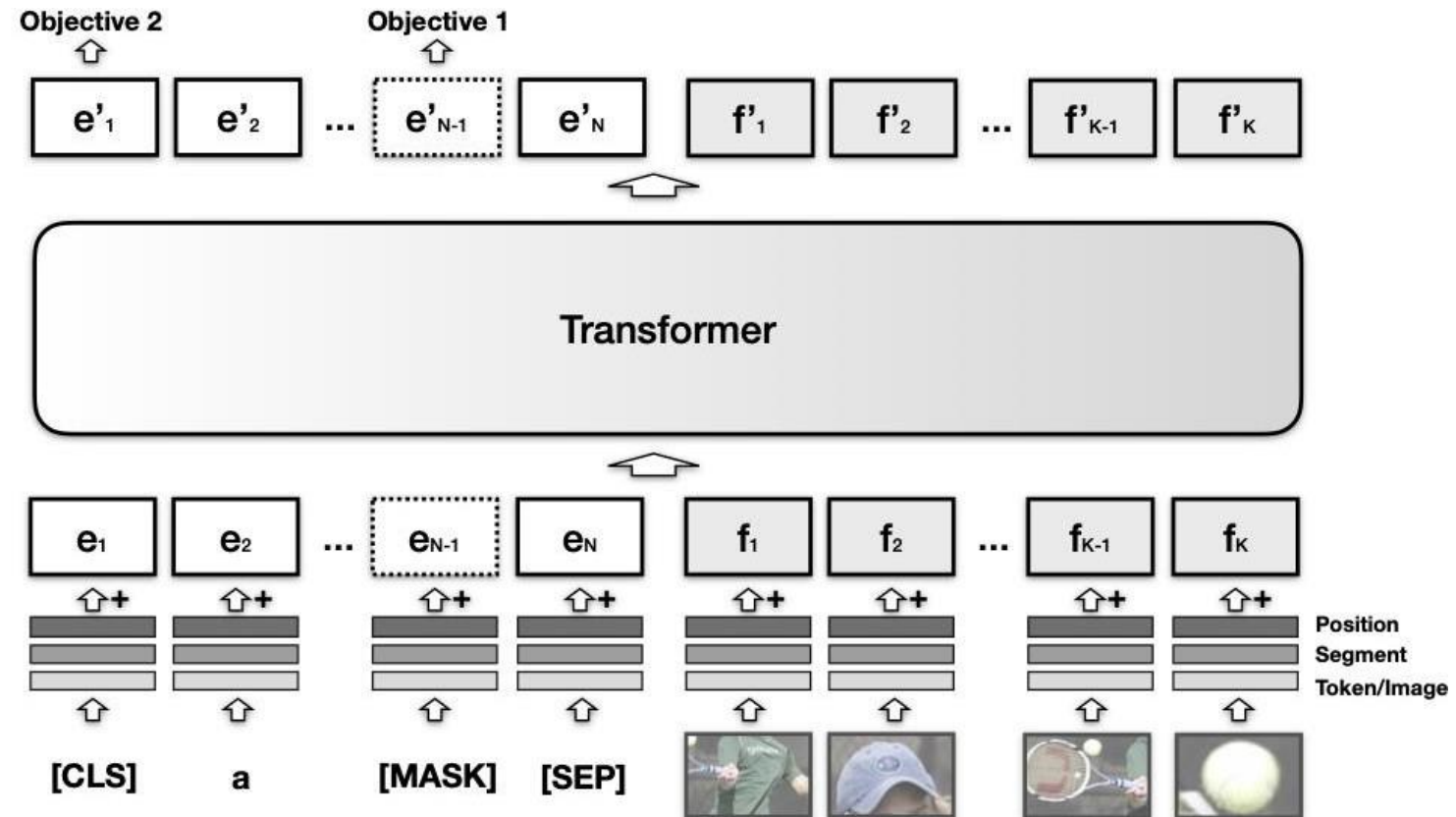


Option 1: Concatenate modalities and learn BERT transformer

VisualBERT



A person hits a ball with a tennis racket



Li, Liunian Harold, et al. "Visualbert: A simple and performant baseline for vision and language. (2019)

Recall: Cross-Attention

- Given two modalities $a \in \mathbb{R}^{n_A \times d}$, $b \in \mathbb{R}^{n_B \times d}$
- The attention score is based on a **dot-product**
 - $Q_A = a \cdot W_{A,Q} \in \mathbb{R}^{n_A \times d}$
 - $K_B = b \cdot W_{B,K} \in \mathbb{R}^{n_B \times d}$
 - Where: $W_{A,Q} \in \mathbb{R}^{d \times d}$, $W_{B,K} \in \mathbb{R}^{d \times d}$

Correlation Matrix:

$$\begin{matrix} & Q_A \\ & \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \\ & \times \\ & \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline \end{array} K_B^T \\ \in \mathbb{R}^{n_A \times n_B} \end{matrix}$$

Cross-attention correlation matrix

$$Q_A \times K_B^T \in \mathbb{R}^{n_A \times n_B}$$


Correlation Matrix:

			
What	1.2	2.3	1.7
animal	-0.4	6.5	4.2
is	1.3	1.2	0.6

Cross-attention for multimodal

1)

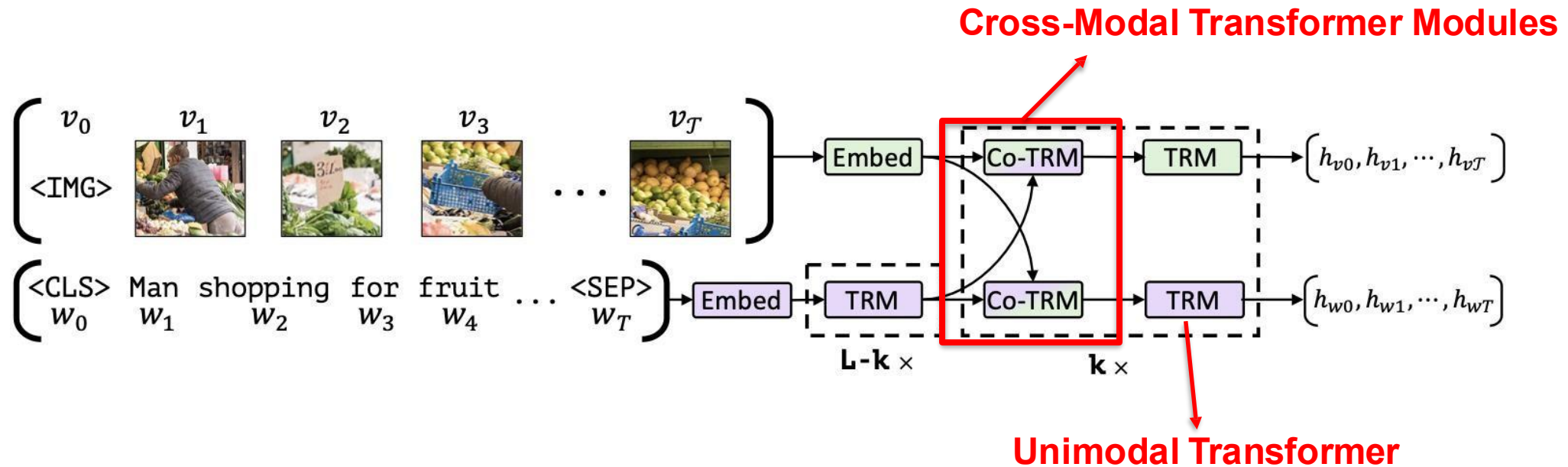
$$\begin{aligned}
 & \text{softmax} \left(\frac{\overbrace{Q_A \times K_B^T}^{\mathbb{R}^{n_A \times n_B}}}{\sqrt{d_k}} \right) V_B \\
 & \quad \mathbb{R}^{n_B \times d} \\
 & \quad V_B \\
 & = Z_{B \rightarrow A} \\
 & \quad \mathbb{R}^{n_A \times d}
 \end{aligned}$$

2)

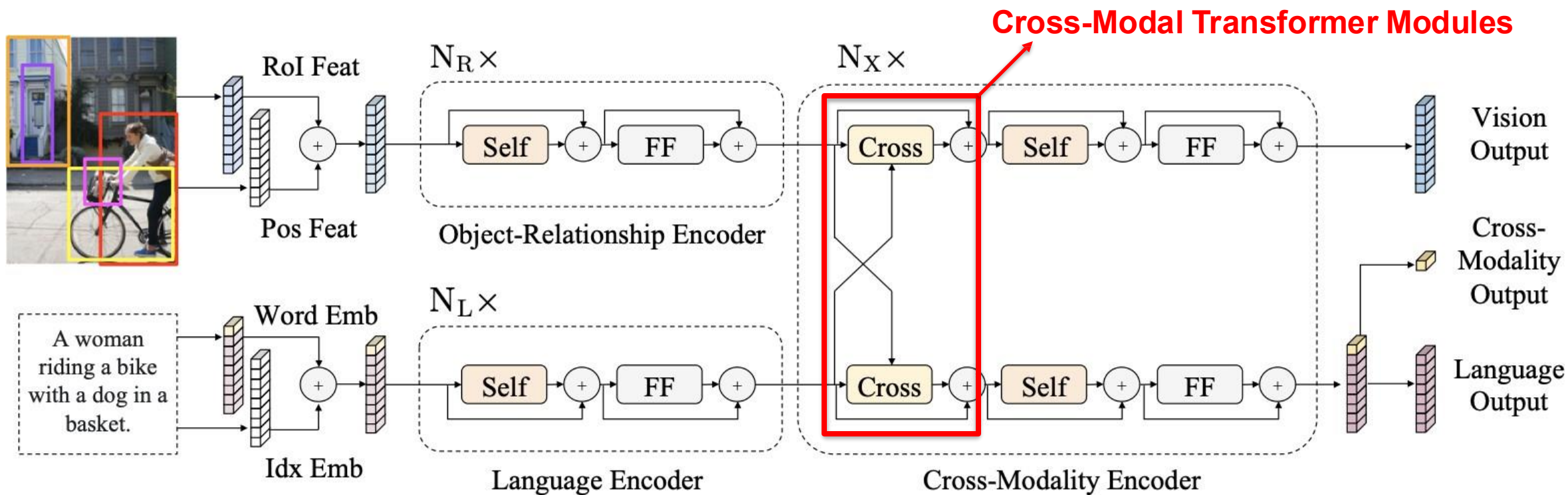
Residual connection

$$Q_A + Z_{B \rightarrow A}$$

ViLBERT

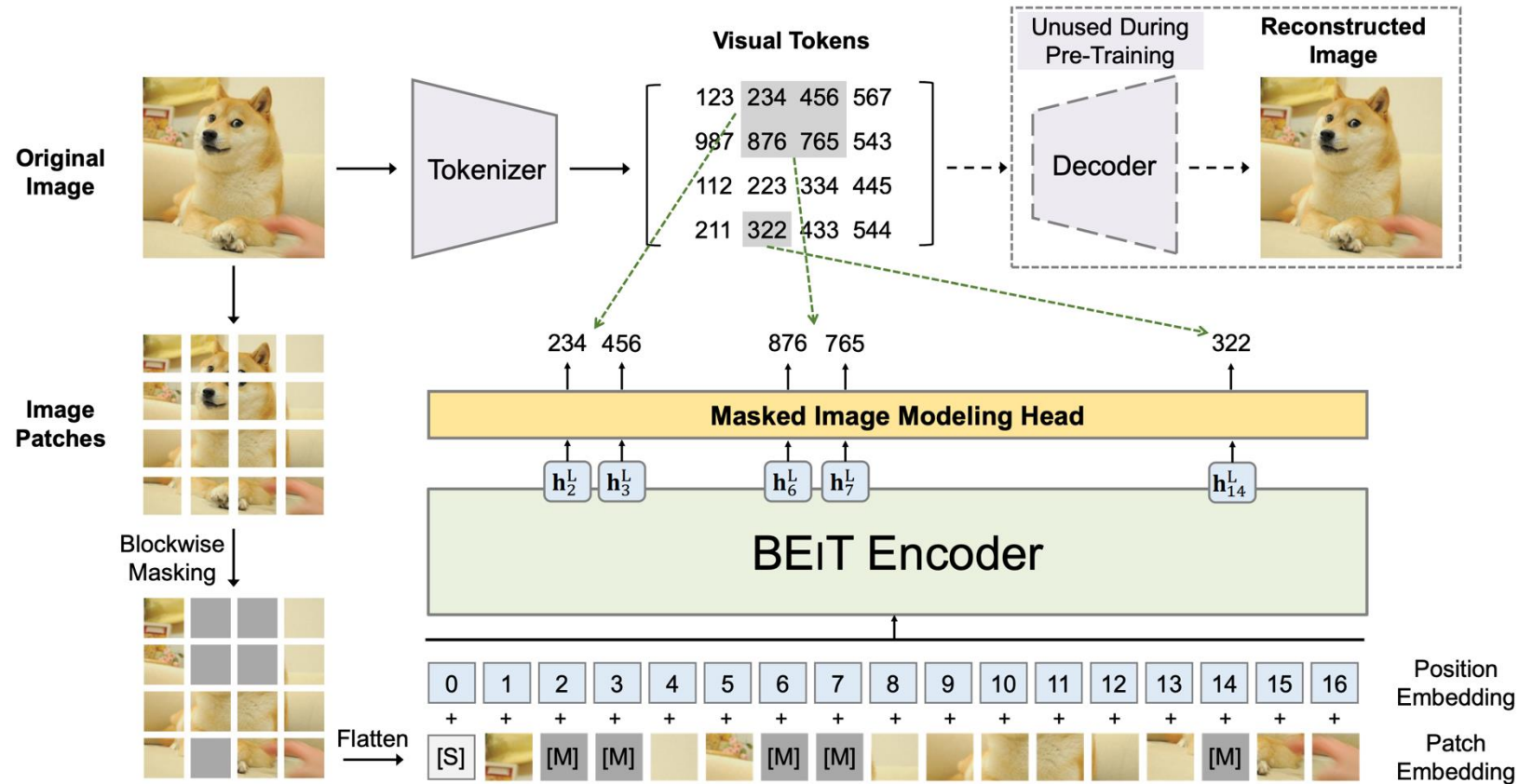


LXMERT



Tan, Hao, and Mohit Bansal. "Lxmert: Learning cross-modality encoder representations from transformers." *arXiv* (August 20, 2019).

Multimodal Self-supervised: Visual Tokens

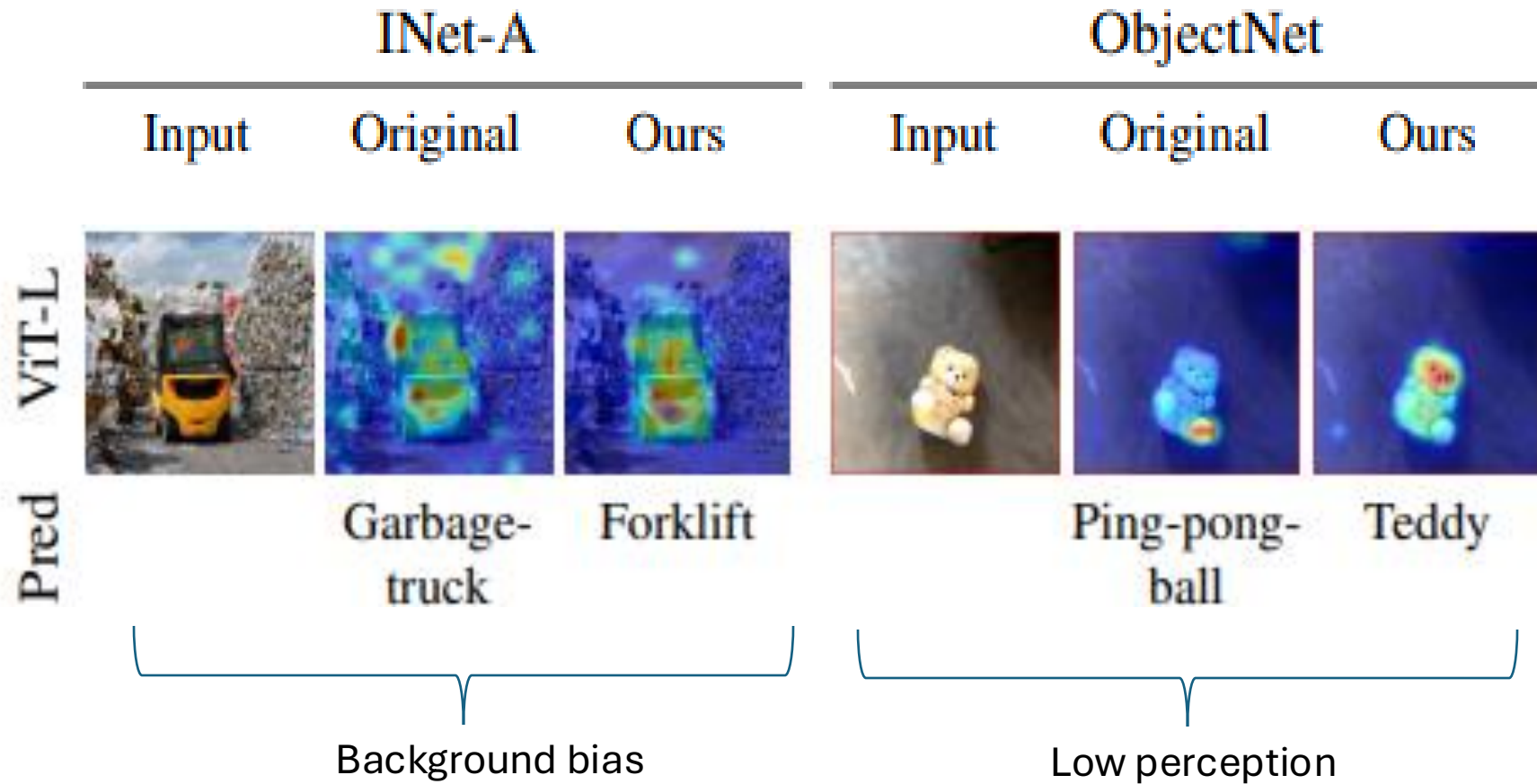


Appendix

Optimizing Relevance maps

Image self-attention

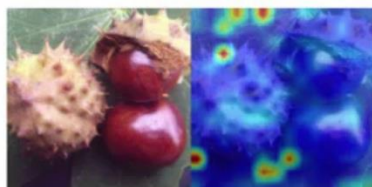
Issues with Transformer



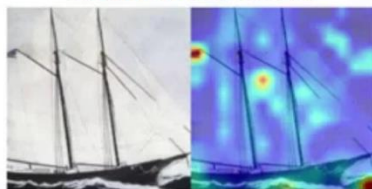
Problem Definition

- Vision models tend to rely on **spurious cues** to make predictions
- We find that Vision Transformers often suffer from two main salient issues:
 - Over reliance on the background
 - Sparse consideration of the foreground pixels

ViT-base [15]



Classification: Chestnut
Confidence: 100%

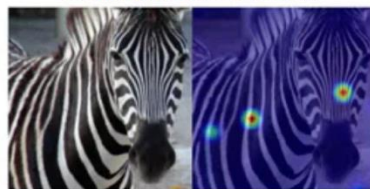


Classification: Schooner
Confidence: 97%

AR-base [54]



Classification: Bullfrog
Confidence: 99.1%

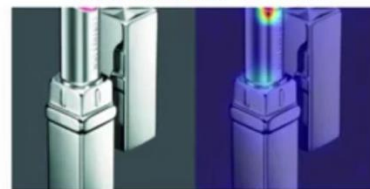


Classification: Zebra
Confidence: 99.9%

DeiT-base [57]



Classification: Dishcloth
Confidence: 91.6%

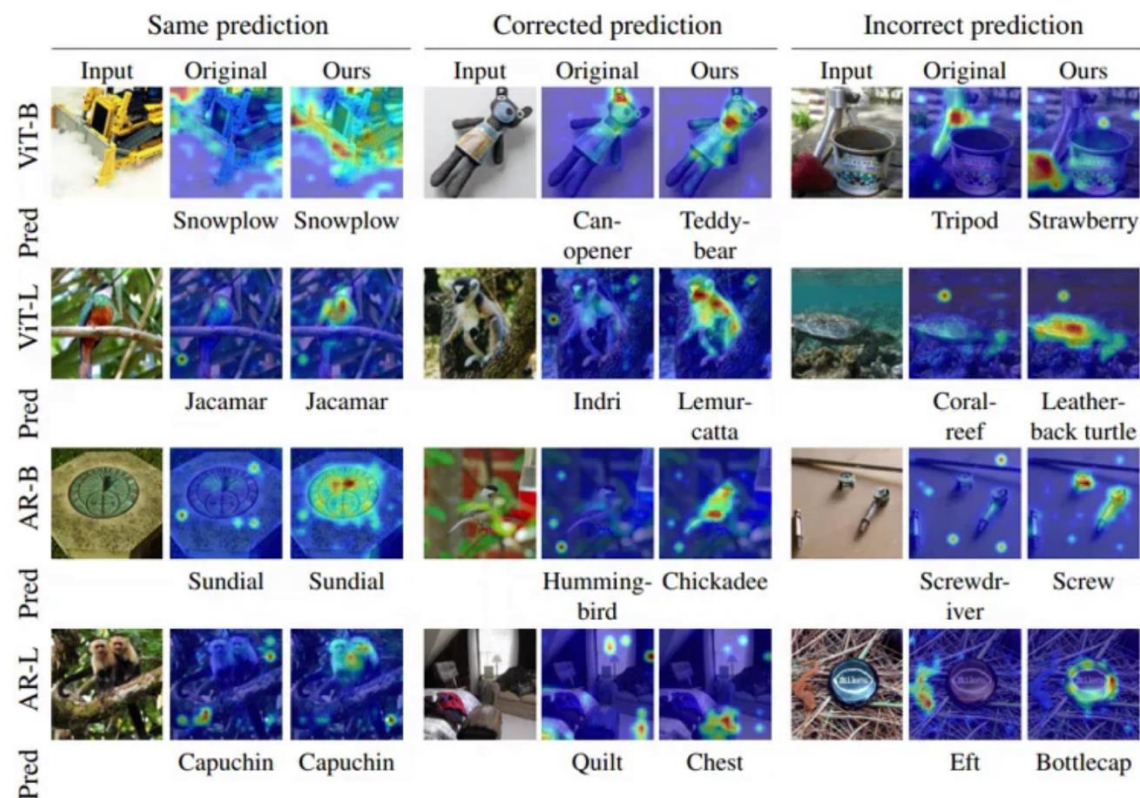


Classification: Lipstick
Confidence: 96.1%

- This leads to lack of robustness to distribution shifts, i.e., **poor generalization**
 - Can we correct the model's logic?

Method intuition

- Directly manipulate the relevance maps to solve both issues
 - Less relevance on the background
 - More relevance on the **entire foreground**



Optimizing Relevance Maps of Vision Transformers Improves Robustness*

- Employ segmentation to train the model where to look



Don't look on the background

$$\mathcal{L}_{\text{bg}} = \text{MSE} (\mathbf{R}(i) \odot \bar{\mathbf{S}}(i), 0)$$

Look on the entire object

$$\mathcal{L}_{\text{fg}} = \text{MSE} (\mathbf{R}(i) \odot \mathbf{S}(i), 1)$$

Method overview

- Our method employs the losses during a short **finetuning** process on models **pretrained** on ImageNet.
- We add a **classification loss** ($\mathcal{L}_{classification}$) to ensure that the accuracy of the model does not decrease due to the finetuning process.

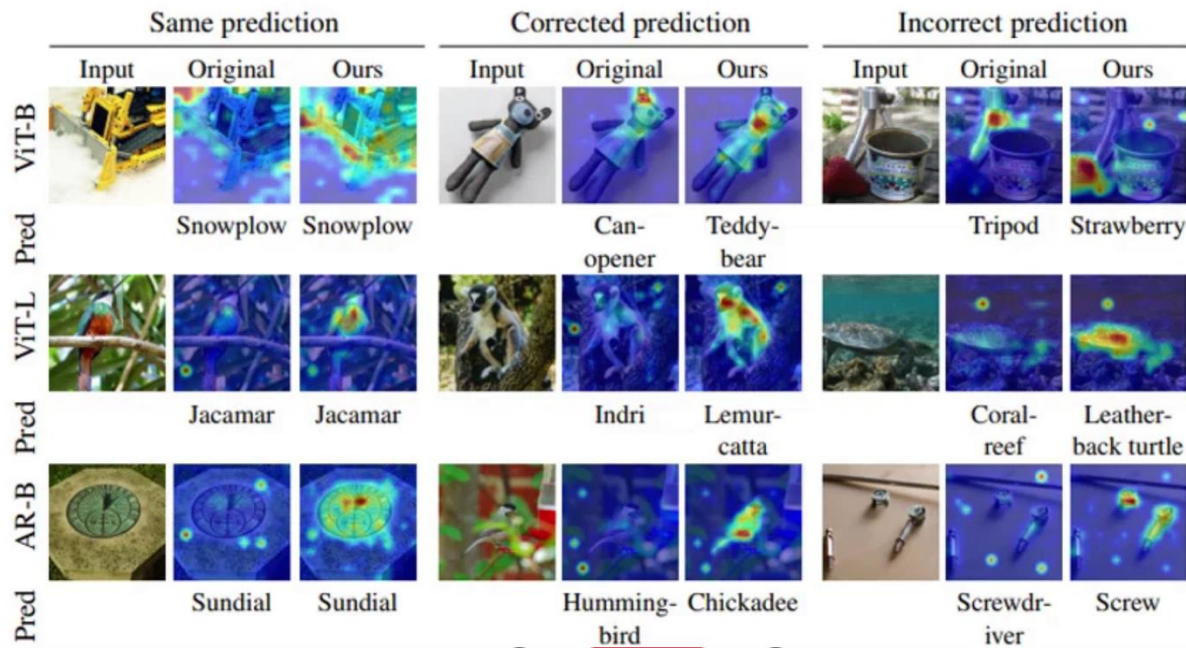
$$\mathcal{L}_{classification} = \text{CE}(\mathcal{M}(i), \arg \max(\mathcal{M}(i)))$$

- Thus, our final loss term is as follows:

$$\mathcal{L} = \lambda_{\text{relevance}} \cdot \mathcal{L}_{\text{relevance}} + \lambda_{\text{classification}} \cdot \mathcal{L}_{\text{classification}}$$

Method overview

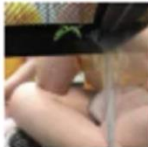
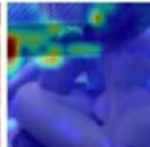
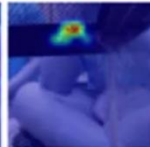
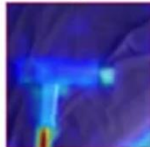
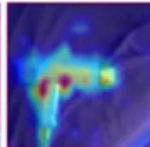
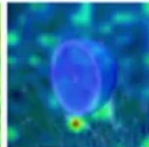
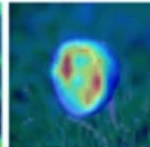
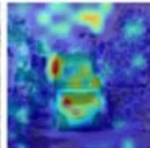
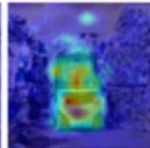
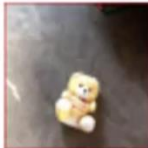
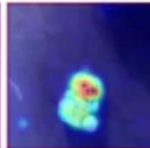
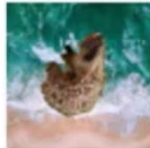
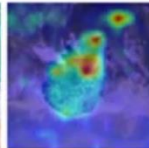
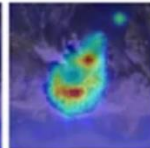
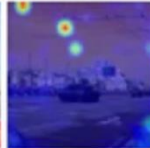
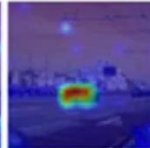
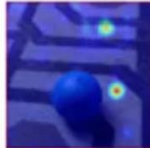
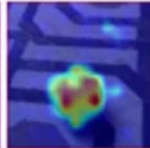
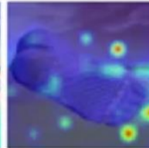
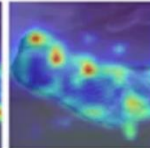
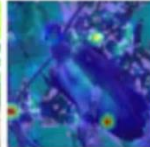
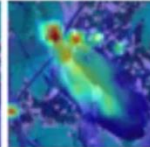
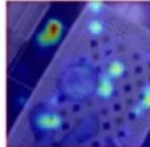
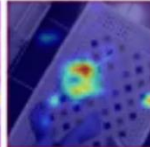
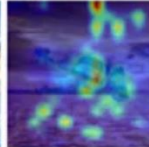
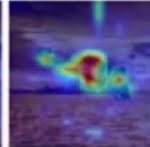
- Our method requires segmentation maps to distinguish between the background ($\mathcal{L}_{bg}$) and the foreground ($\mathcal{L}_{fg}$) of the image.
 - We use either human annotated maps or maps produced by DINO (without human supervision)
- Our method only uses 3 examples from half the ImageNet classes (500 classes) during the finetuning process (overall only 1500 training samples).



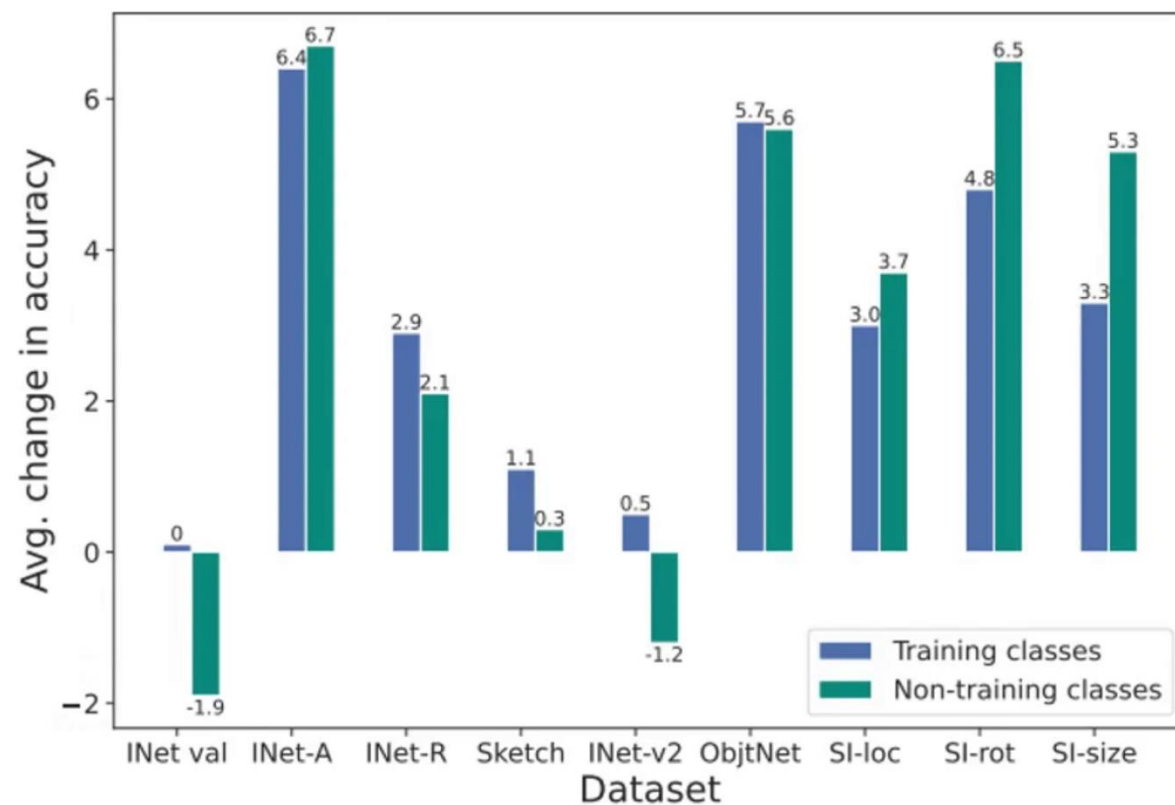
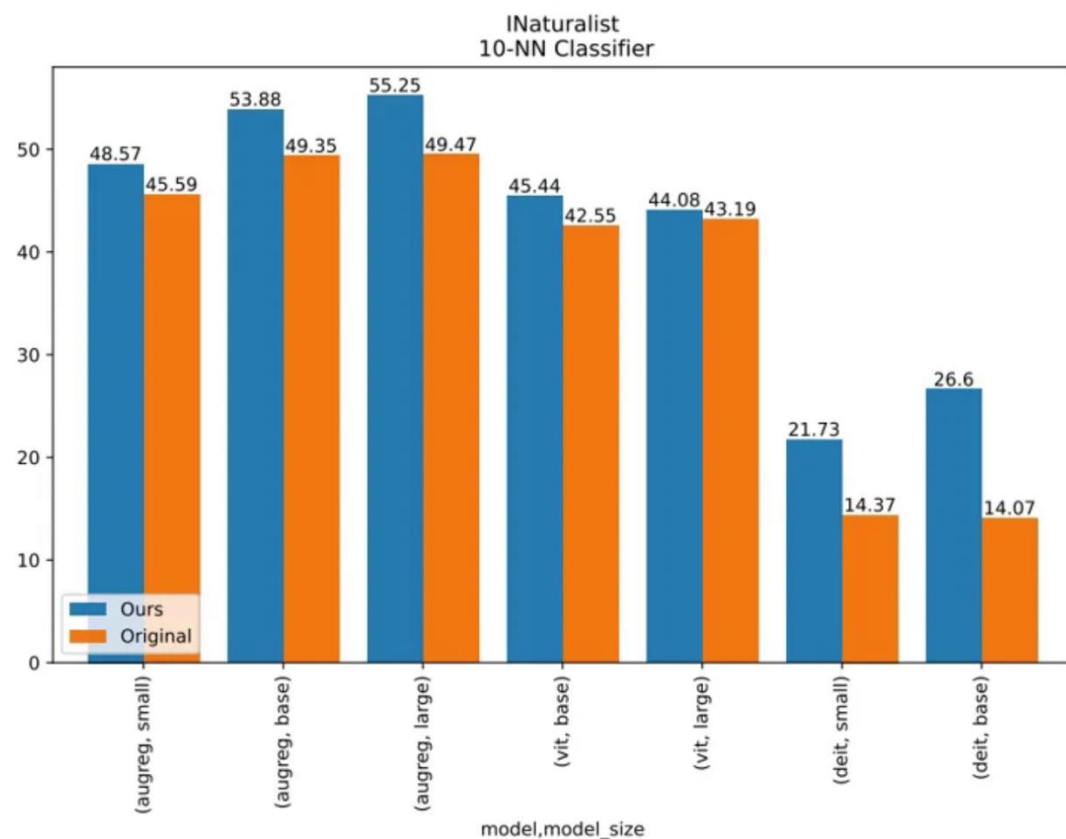
Results

- We test on 3 different types of Vision Transformers- Vanilla ViT, DeiT, and ViT with improved augmentations and regularization (ViT-AR).
- We measure the increase in robustness to distribution shifts, i.e., the increase in accuracy for datasets with shifted distributions.
 - We use adversarial, real-world, and synthetic datasets and observe a significant increase in robustness in all cases.

Results

INet-A			ObjectNet			SI-rotation			
	Input	Original	Ours	Input	Original	Ours	Input	Original	Ours
ViT-B									
		Television	Mantis		Drill	Hand-blower		Golf ball	Lemon
ViT-L									
		Garbage-truck	Forklift		Ping-pong-ball	Teddy		Sea Lion	Porcupine
AR-B									
		Tram	Tank		Maze	Orange		Swimming trunks	Sock
AR-L									
		Kite	Vulture		Scale	Strainer		Lifeboat	Cello

Results

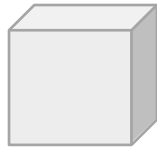


Example: CNN with Self-Attention

Input Image

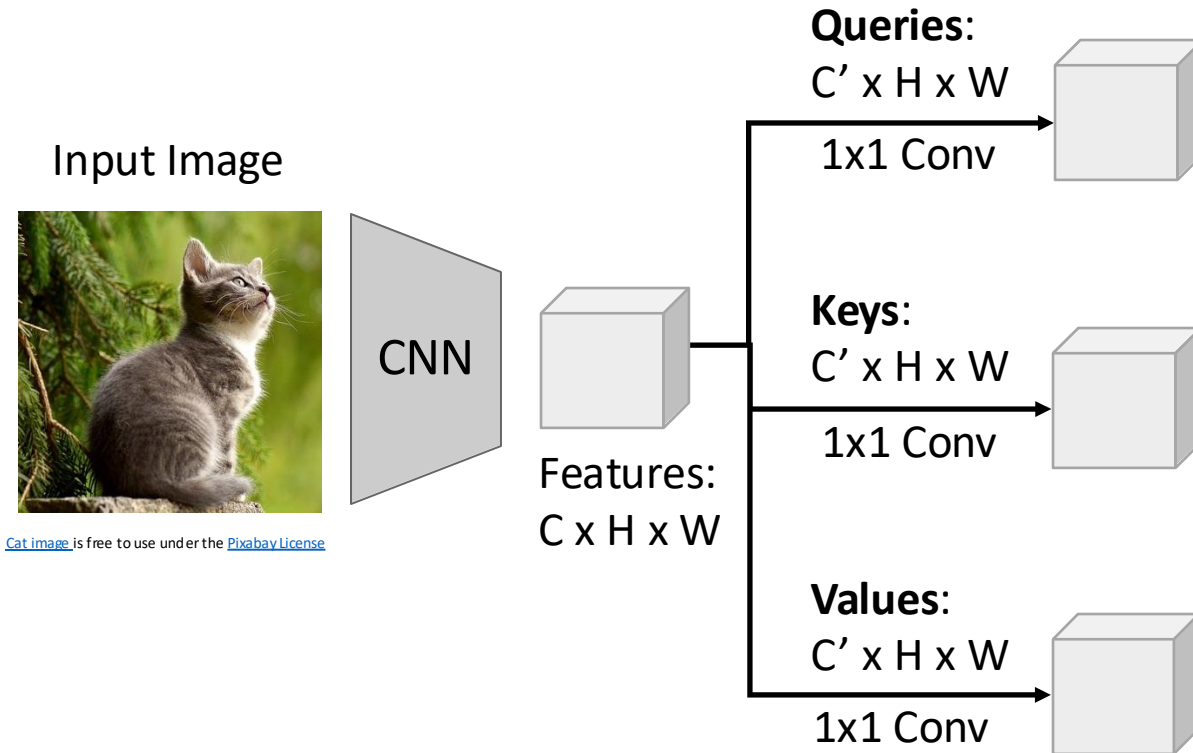


[Cat image](#) is free to use under the [Pixabay License](#)

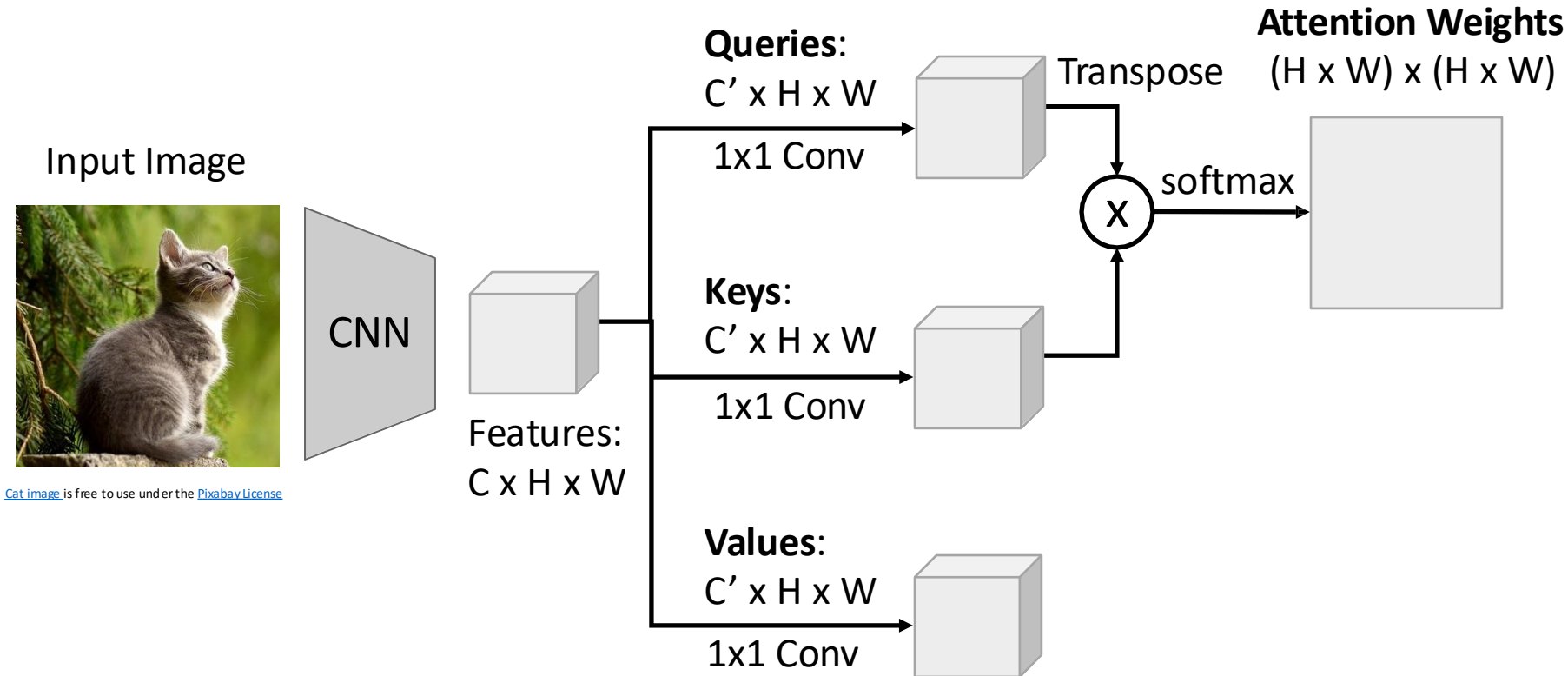


Features:
 $C \times H \times W$

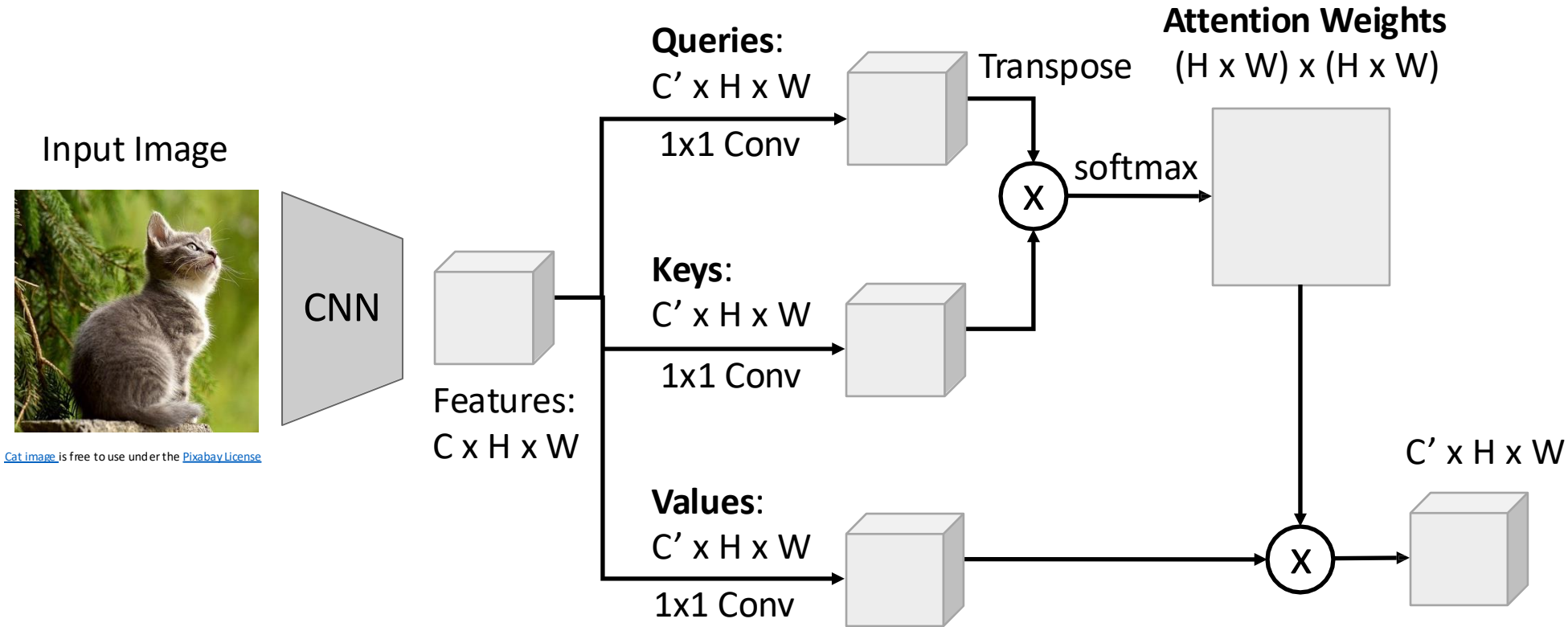
Example: CNN with Self-Attention



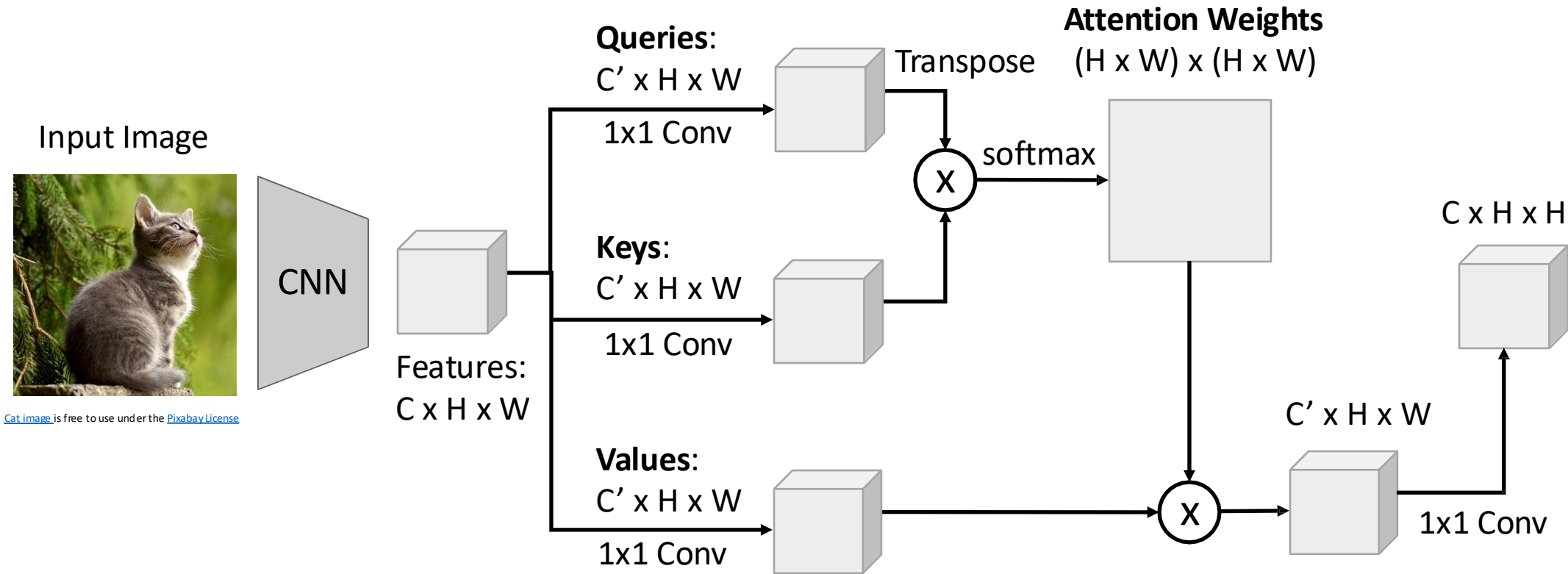
Example: CNN with Self-Attention



Example: CNN with Self-Attention



Example: CNN with Self-Attention



Example: CNN with Self-Attention



[Cat image](#) is free to use under the [Pixabay License](#)

